



# Basi di Dati - Modulo Laboratorio A.a. 2025-2026

## Lezione 8: MongoDB

Dr. Beatrice Amico



# NoSQL Systems – MongoDB

MongoDB è un sistema per la gestione di basi di dati NoSQL di tipo document-based.

- Una base di dati è formata da **collezioni di documenti**.
- Ciascuna **collezione** corrisponde al concetto di **tabella** (o entità).
- Ciascuna collezione è formata da insiemi di **documenti** (**tuple**).

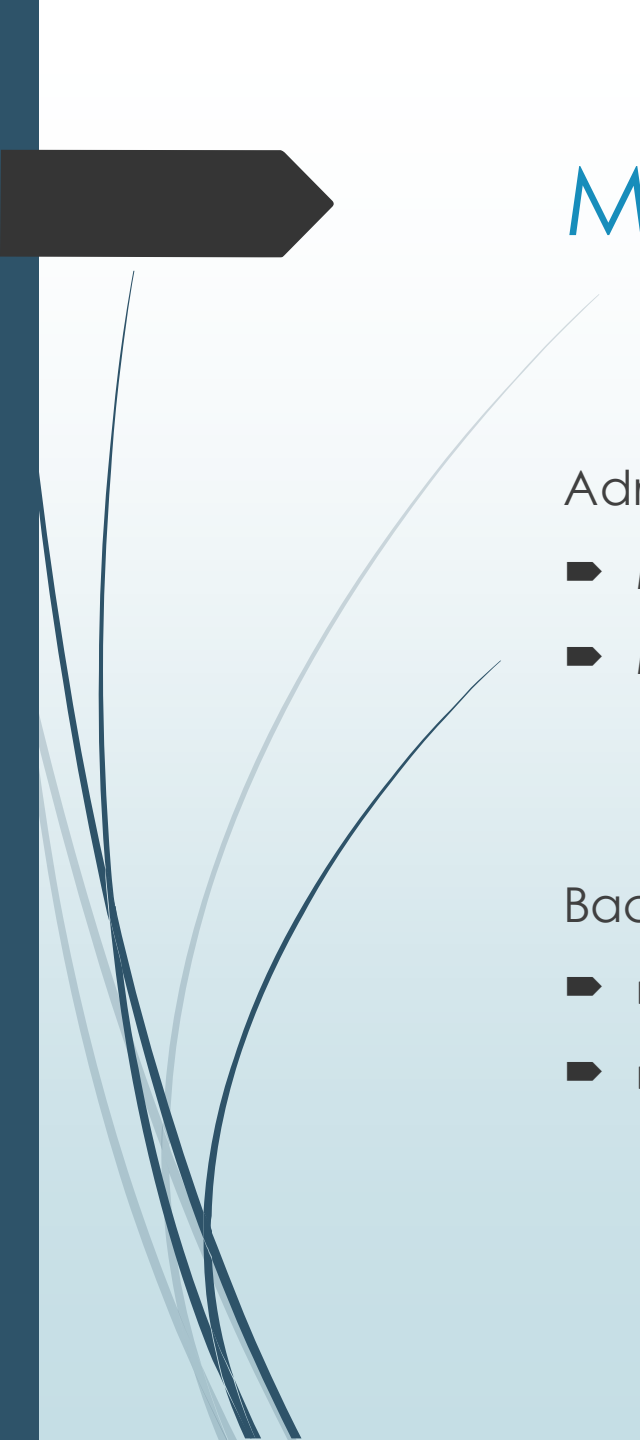
Altre tipologie di database NoSQL

- Graph database (e.g. Neo4j)
- Column-oriented database (e.g. Apache Cassandra)
- Key-value database (e.g. Amazon DynamoDB)



# MongoDB: Caratteristiche

- Struttura a documenti che ricorda l'organizzazione ad oggetti dei linguaggi di programmazione moderni
- Flessibilità e assenza di schema (schema-less)
  - Dati la cui struttura evolve nel tempo
  - Dati non strutturati o semi-strutturati
- Scalabilità e performance
  - Fondamentalmente MongoDB è un **sistema distribuito**
  - La parte server è stata sviluppata pensando che la configurazione di default prevedesse un **cluster (replica)**
    - Ridondanza e alta disponibilità
- Linguaggio di interrogazione e indici per semplificare e velocizzare operazioni comuni, come `findOne()` e `updateOne()`



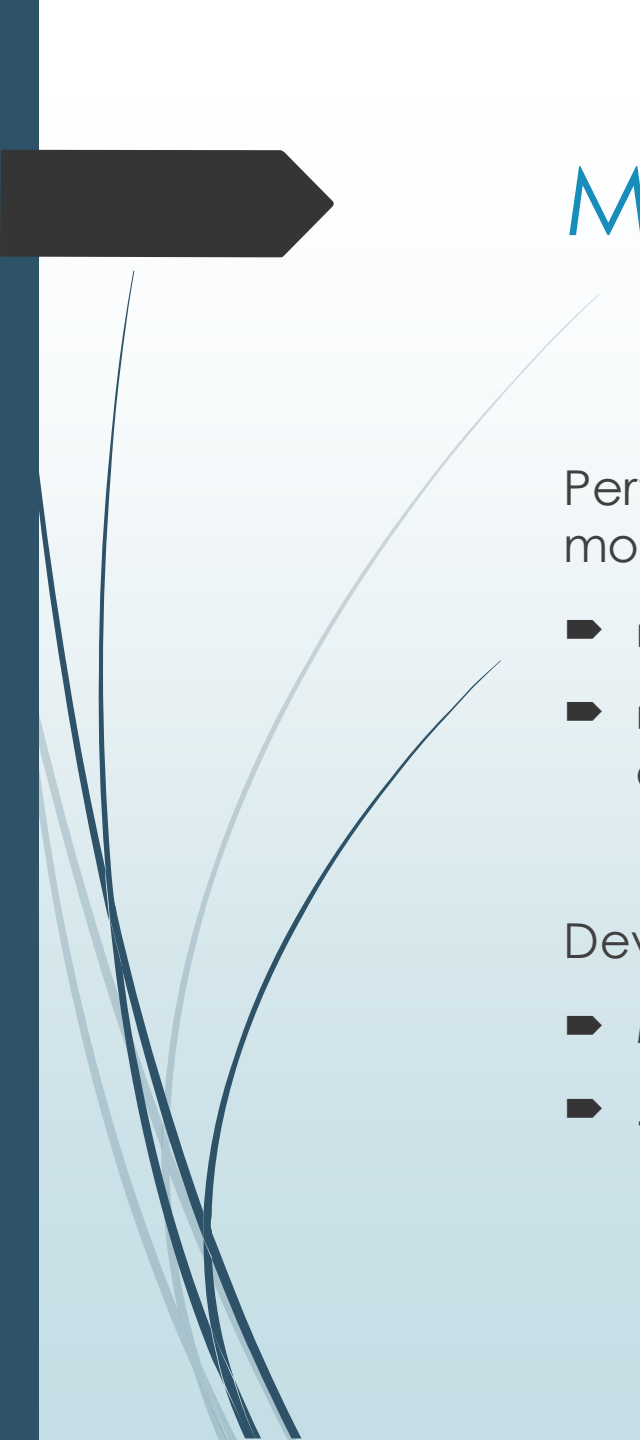
# MongoDB: Tools

Administrative tools: per la gestione dei database

- MongoDB shell (mongosh): da riga di comando
- MongoDB Compass: interfaccia grafica
  - <https://www.mongodb.com/try/download/compass>

Backup e restore: per la creazione o ripristino di backup

- mongodbdump: per creare un backup binario
- mongodbrestore: per il ripristino di un backup



# MongoDB: Tools

Performance and monitoring tools: per verificare le prestazioni del cluster e monitorare lo stato di salute del sistema

- mongostat: statistiche sulle prestazioni
- mongotop: monitorare il tempo richiesta dalle istanze per eseguire operazioni di lettura e scrittura

Developer and integration tools

- MongoDB per Visual Studio Code
- ...

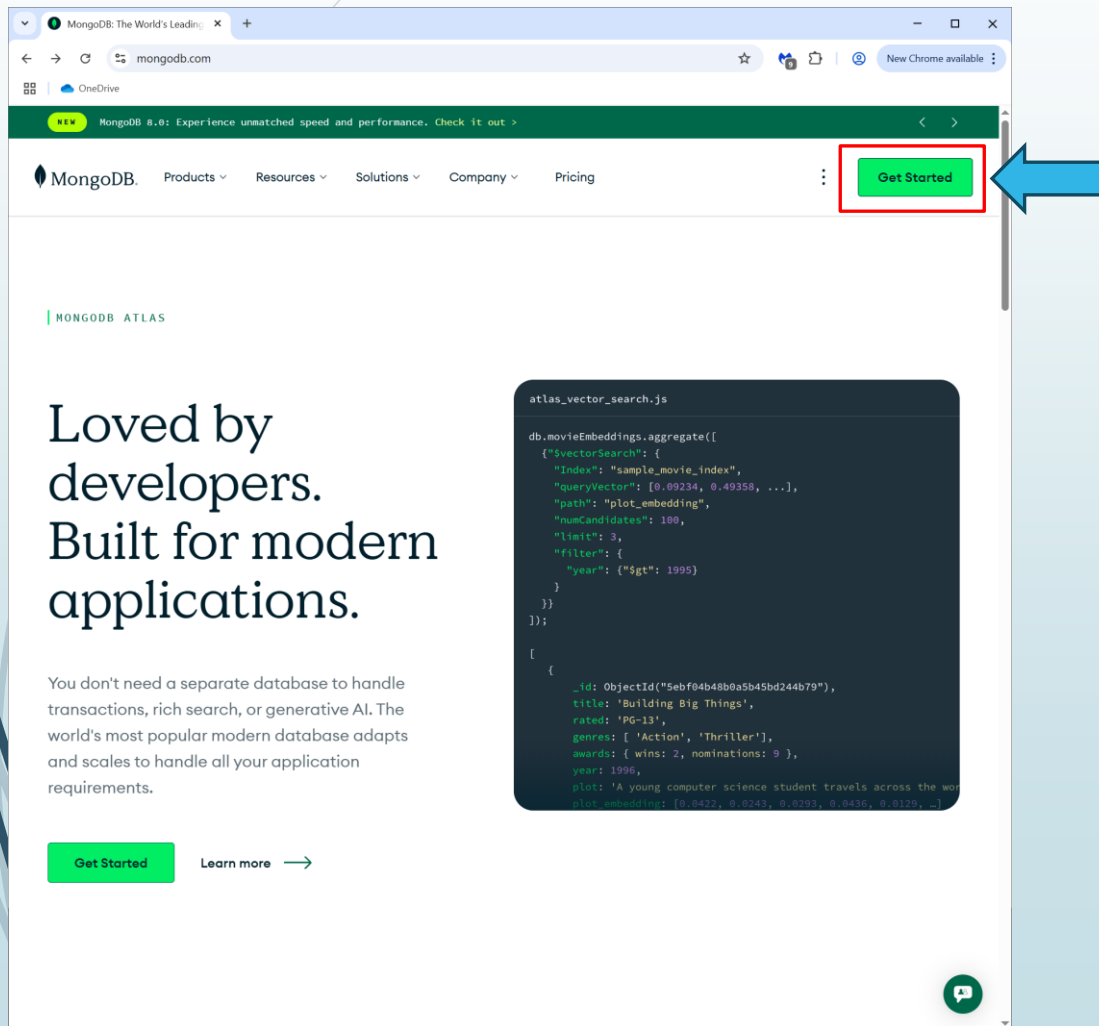


# MongoDB: Installazione

MongoDB cluster:

- Scaricare ed installare MongoDB Community Edition
  - <https://www.mongodb.com/try/download/community>
- Creare un account su MongoDB Atlas ed utilizzare il cluster gratuito
  - <https://account.mongodb.com/account/login?nds=true>

# MongoDB: Cluster su Atlas



MongoDB 8.0: Experience unmatched speed and performance. Check it out >

MongoDB. Products ▾ Resources ▾ Solutions ▾ Company ▾ Pricing

⋮ [Get Started](#)

MONGODB ATLAS

## Loved by developers. Built for modern applications.

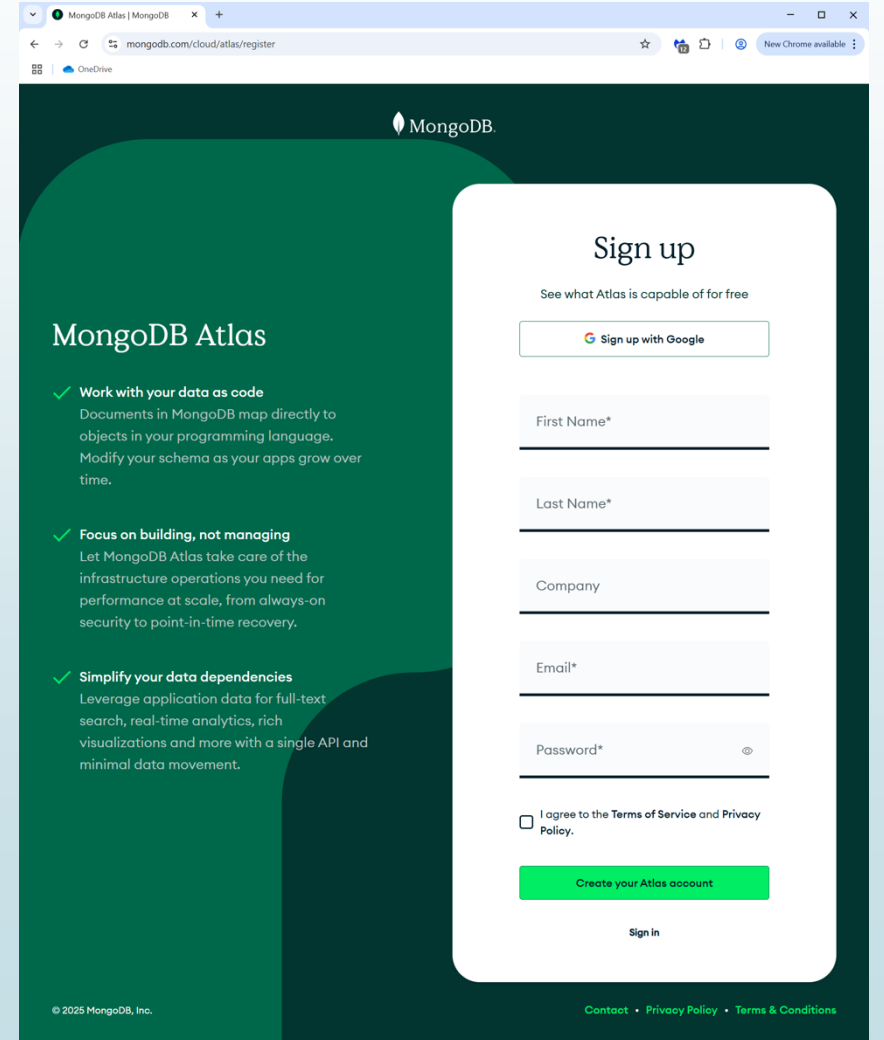
You don't need a separate database to handle transactions, rich search, or generative AI. The world's most popular modern database adapts and scales to handle all your application requirements.

[Get Started](#) [Learn more](#) →

```
atlas_vector_search.js

db.movieEmbeddings.aggregate([
  { "$vectorSearch": {
    "index": "sample_movie_index",
    "queryVector": [0.09234, 0.49358, ...],
    "path": "plot_embedding",
    "numCandidates": 100,
    "limit": 3,
    "filter": {
      "year": { "$gt": 1995 }
    }
  }}
]);

[
  {
    "_id": ObjectId("5ebf84b48b0a5b45bd244b79"),
    "title": "Building Big Things",
    "rated": "PG-13",
    "genres": [ "Action", "Thriller" ],
    "awards": { wins: 2, nominations: 9 },
    "year": 1996,
    "plot": "A young computer science student travels across the world to find a missing person.",
    "plot_embedding": [0.0432, 0.0245, 0.0293, 0.0436, 0.0129, ...]
  }
]
```



MongoDB

## Sign up

See what Atlas is capable of for free

[Sign up with Google](#)

First Name\*

Last Name\*

Company

Email\*

Password\*

☐ I agree to the [Terms of Service](#) and [Privacy Policy](#).

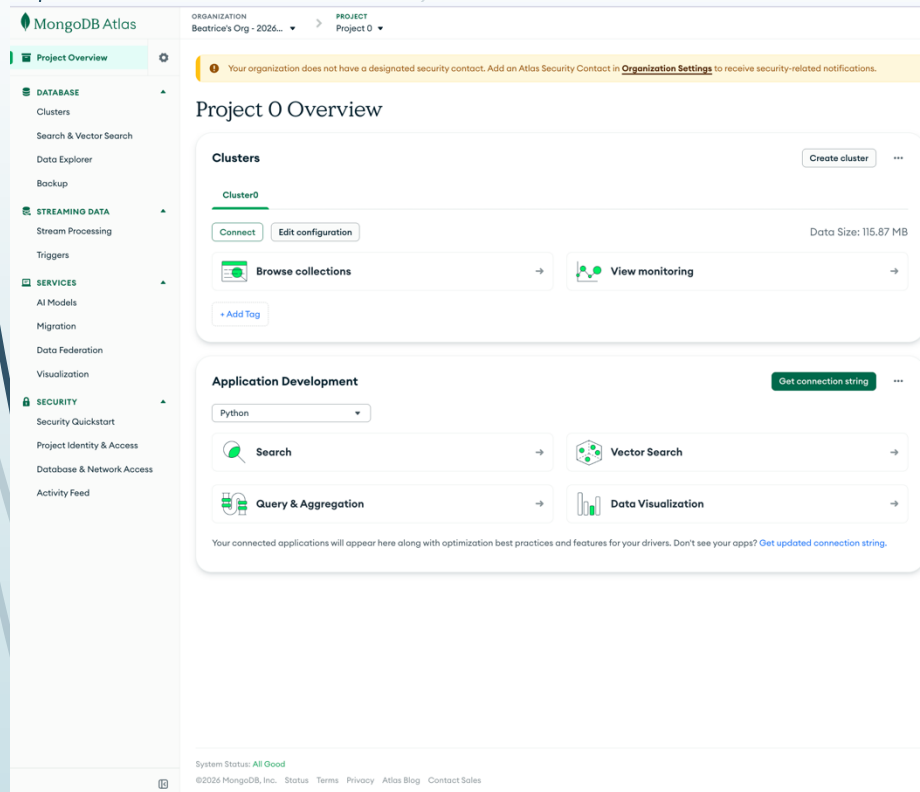
[Create your Atlas account](#)

[Sign in](#)

© 2025 MongoDB, Inc. [Contact](#) • [Privacy Policy](#) • [Terms & Conditions](#)

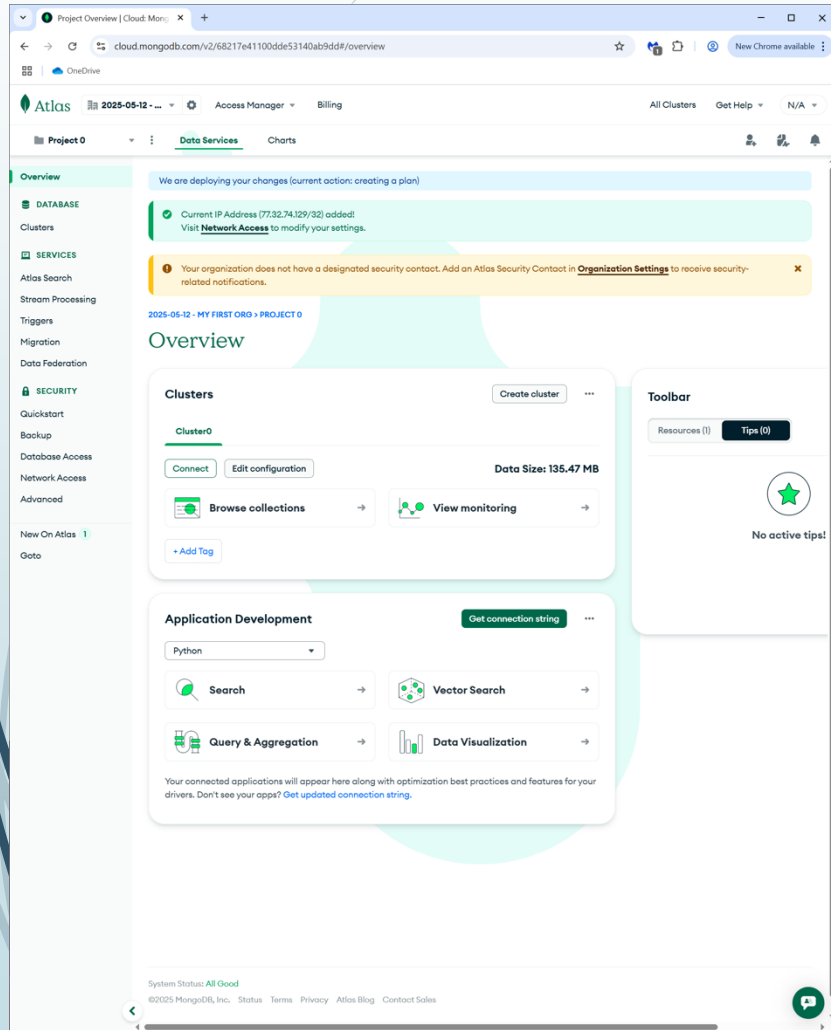
- ✓ **Work with your data as code**  
Documents in MongoDB map directly to objects in your programming language. Modify your schema as your apps grow over time.
- ✓ **Focus on building, not managing**  
Let MongoDB Atlas take care of the infrastructure operations you need for performance at scale, from always-on security to point-in-time recovery.
- ✓ **Simplify your data dependencies**  
Leverage application data for full-text search, real-time analytics, rich visualizations and more with a single API and minimal data movement.

# MongoDB: Creazione Cluster su Atlas



1. Aggiungere l'IP alla lista di quelli autorizzati alla connessione

# MongoDB: Creazione Cluster su Atlas

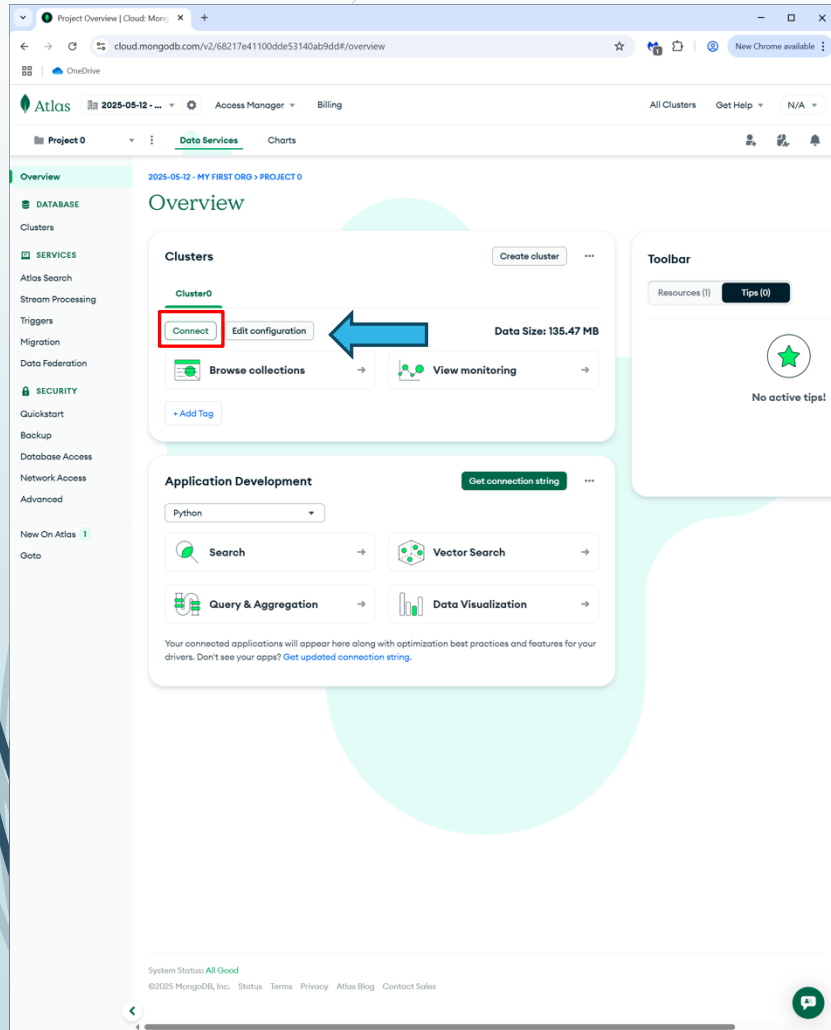


1. Aggiungere l'IP alla lista di quelli autorizzati alla connessione



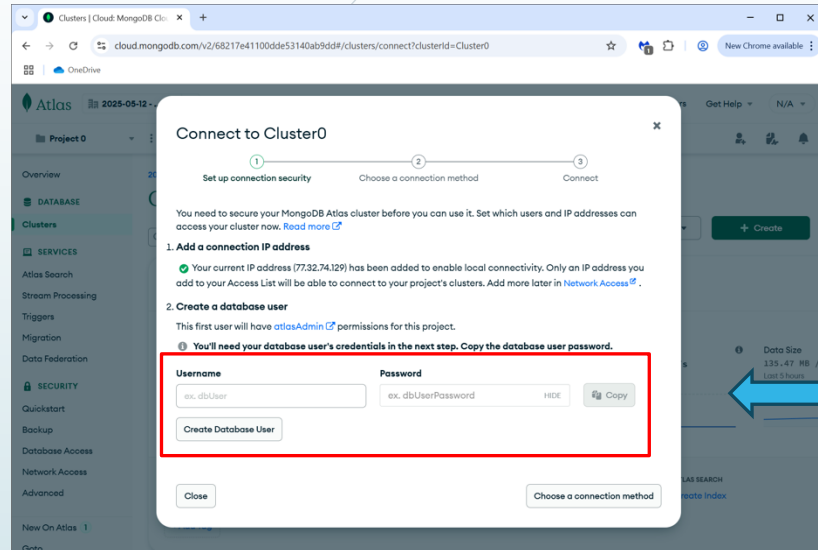
2. Completare le configurazioni di sicurezza aggiungendo un'email di contatto

# MongoDB: Creazione Cluster su Atlas



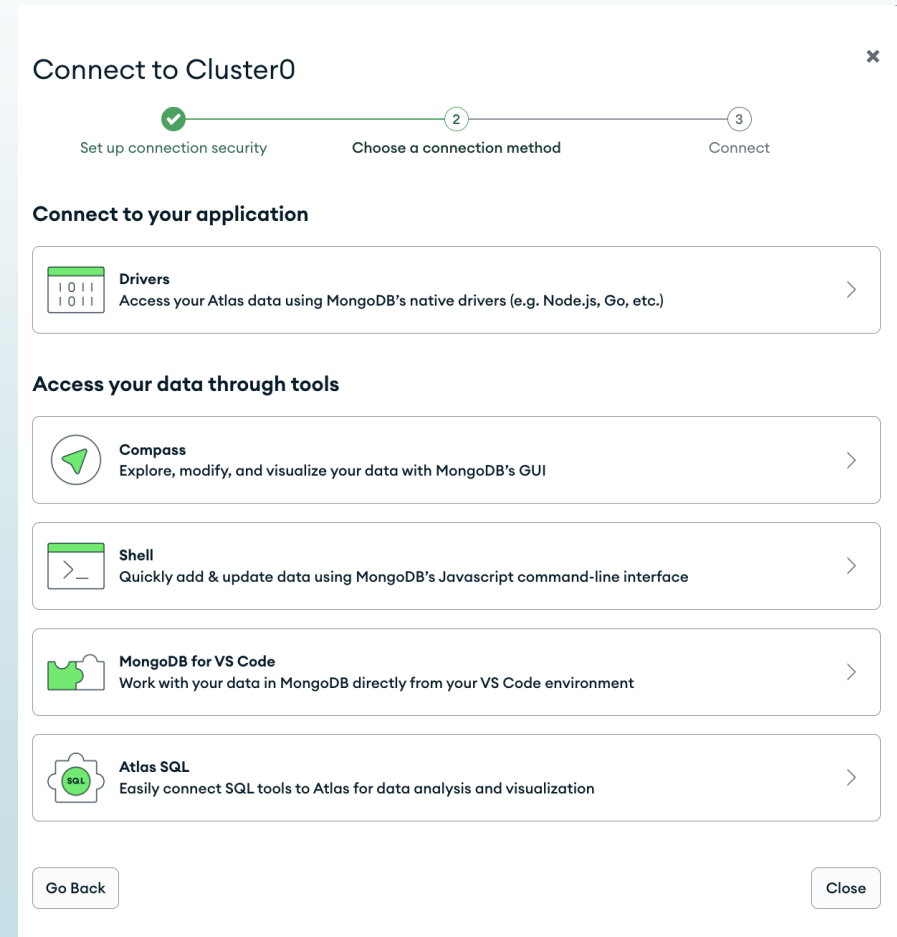
1. Aggiungere l'IP alla lista di quelli autorizzati alla connessione
2. Completare le configurazioni di sicurezza aggiungendo un'email di contatto
3. Connessione

# MongoDB: Creazione DB e Connessione



3a. Creazione utente

3b. Scelta del client



# Connessione a MongoDB tramite mongosh

Connect to Cluster0

✓

✓

3

Set up connection securityChoose a connection methodConnect

Connecting with MongoDB Driver

1. Select your driver and version

We recommend installing and using the latest driver version.

Driver

Version

Python

4.7 or later

2. Install your driver

Run the following on the command line

Note: Use appropriate Python 3 executable

```
python -m pip install "pymongo[srv]"
```

[View MongoDB Python Driver installation instructions.](#)

3. Add your connection string into your application code

Use this connection string in your application

☒ SRV Connection String ?☐ View full code sample

```
mongodb+srv://admin:<db_password>@cluster0.7vninfb.mongodb.net/?appName=Cluster0
```

Replace **<db\_password>** with the password for the **admin** database user. Ensure any option params are [URL encoded](#).

RESOURCES

[Get started with the Python Driver](#)[Python Starter Sample App](#)[Access your Database Users](#)[Troubleshoot Connections](#)

Go Back

Done

# Connessione a MongoDB tramite mongosh

- La shell mongosh può essere installata in vario modo:
  - Download al link: <https://www.mongodb.com/try/download/shell>
  - Usare NPM (Node Package Manager) se Node.js è installato nel Sistema
    - Aprire il terminale
    - `npm install -g mongosh`
- Connessione ad un server MongoDB
  - Comando generico:
  - `mongosh "mongodb://username:password@hostname:port/test"`

Cluster Atlas (copia ed incolla dall'interfaccia web):

```
mongodb+srv://<username>:<db_password>@cluster0.7vninfb.mongodb.net/?appName=Cluster0
```

# Connessione a MongoDB tramite mongosh

```
>_MONGOSH
Atlas atlas-uztavi-shard-0 [primary] test>
```

# Comandi principali di mongosh

Mostrare tutti i database con show dbs

```
>_MONGOSH
> show dbs
< museo      108.00 KiB
  sample_mflix 130.15 MiB
  univrrdb     8.00 KiB
  admin        356.00 KiB
  local        4.67 GiB
Atlas atlas-uztavi-shard-0 [primary] test>
```

# Comandi principali di mongosh

- `use sample_mflix`: Selezionare un particolare database su cui lavorare
- `show collections`: Mostrare le collezioni di documenti presenti in un database

```
>_MONGOSH
> show dbs
< museo          108.00 KiB
  sample_mflix  130.15 MiB
  univrrdb       8.00 KiB
  admin          356.00 KiB
  local          4.67 GiB
> use sample_mflix
< switched to db sample_mflix
> show collections
< comments
  embedded_movies
  movies
  sessions
  theaters
  users
Atlas atlas-uztavi-shard-0 [primary] sample_mflix>
```

# Comandi principali di mongosh

Visualizzare tutti i comandi disponibili

► test> help

Uscire dalla shell

► test> exit

```
> _MONGOSH
> help
< Shell Help

log                'log.info(<msg>)': Write a custom info/warn/error/fatal/debug message to the log file
                  'log.getPath()': Gets a path to the current log file

use                Set current database

show               'show databases'/'show dbs': Print a list of all available databases 'show collections'/'show
                  tables': Print a list of all collections for current database 'show profile': Prints system.profile
                  information 'show users': Print a list of all users for current database 'show roles': Print a list
                  of all roles for current database 'show log <name>': Display log for current connection, if name is
                  not set uses 'global' 'show logs': Print all logger names.

exit               Quit the MongoDB shell with exit/exit()/exit

quit               Quit the MongoDB shell with quit/quit()

Mongo              Create a new connection and return the Mongo object. Usage: new Mongo(URI, options [optional])

connect            Create a new connection and return the Database object. Usage: connect(URI, username [optional],
                  password [optional])

it                 result of the last line evaluated; use to further iterate

version            Shell version

load               Loads and runs a JavaScript file into the current shell environment

enableTelemetry     Enables collection of anonymous usage data to improve the mongosh CLI

disableTelemetry    Disables collection of anonymous usage data to improve the mongosh CLI

passwordPrompt      Prompts the user for a password

sleep              Sleep for the specified number of milliseconds

print              Prints the contents of an object to the output

printjson           Alias for print()

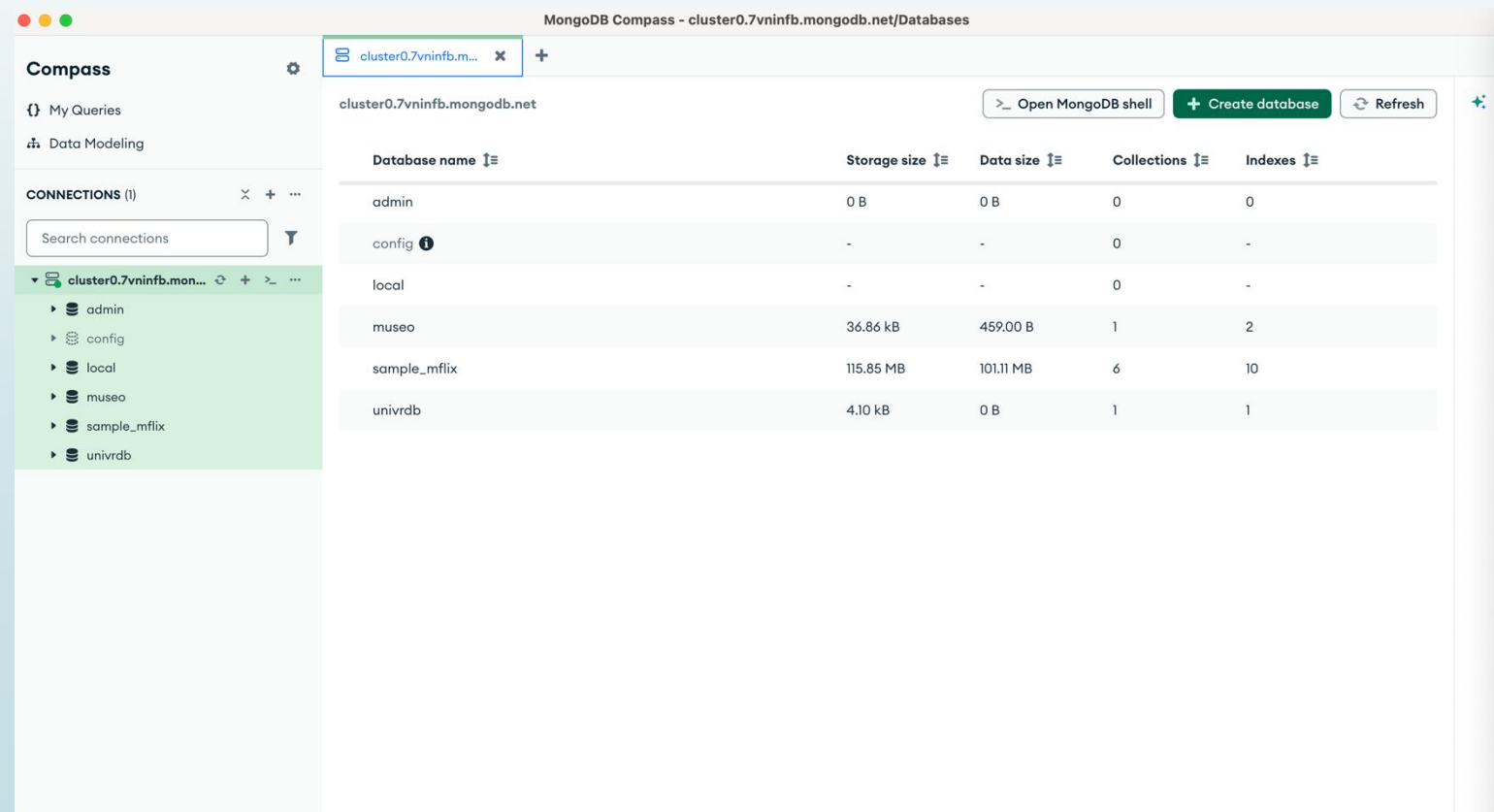
convertShardKeyToHashed Returns the hashed value for the input using the same hashing function as a hashed index.

cls                Clears the screen like console.clear()

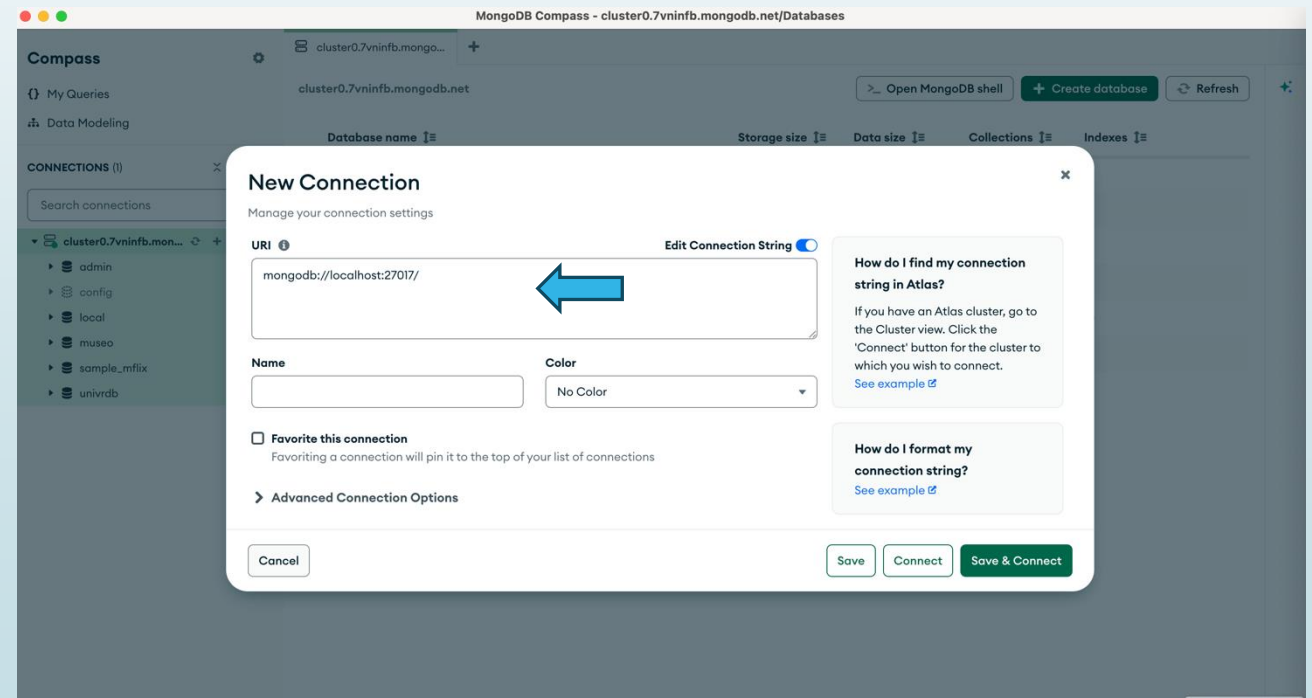
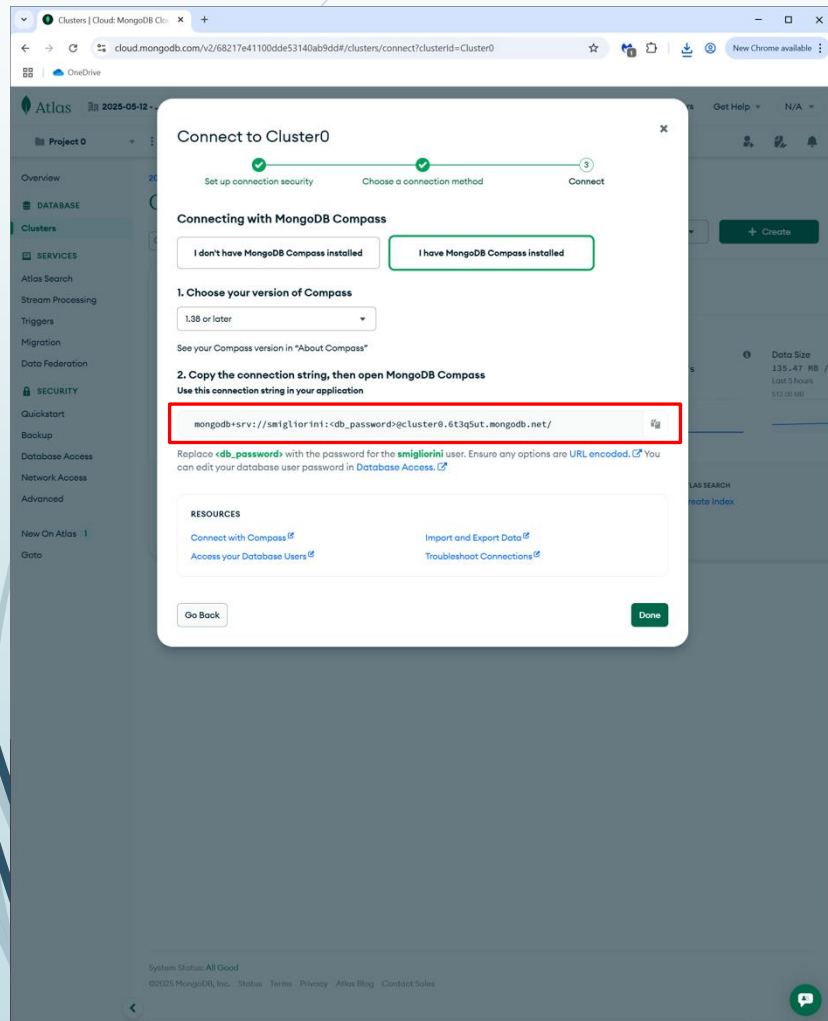
isInteractive        Returns whether the shell will enter or has entered interactive mode
```

# MongoDB Compass: Connessione

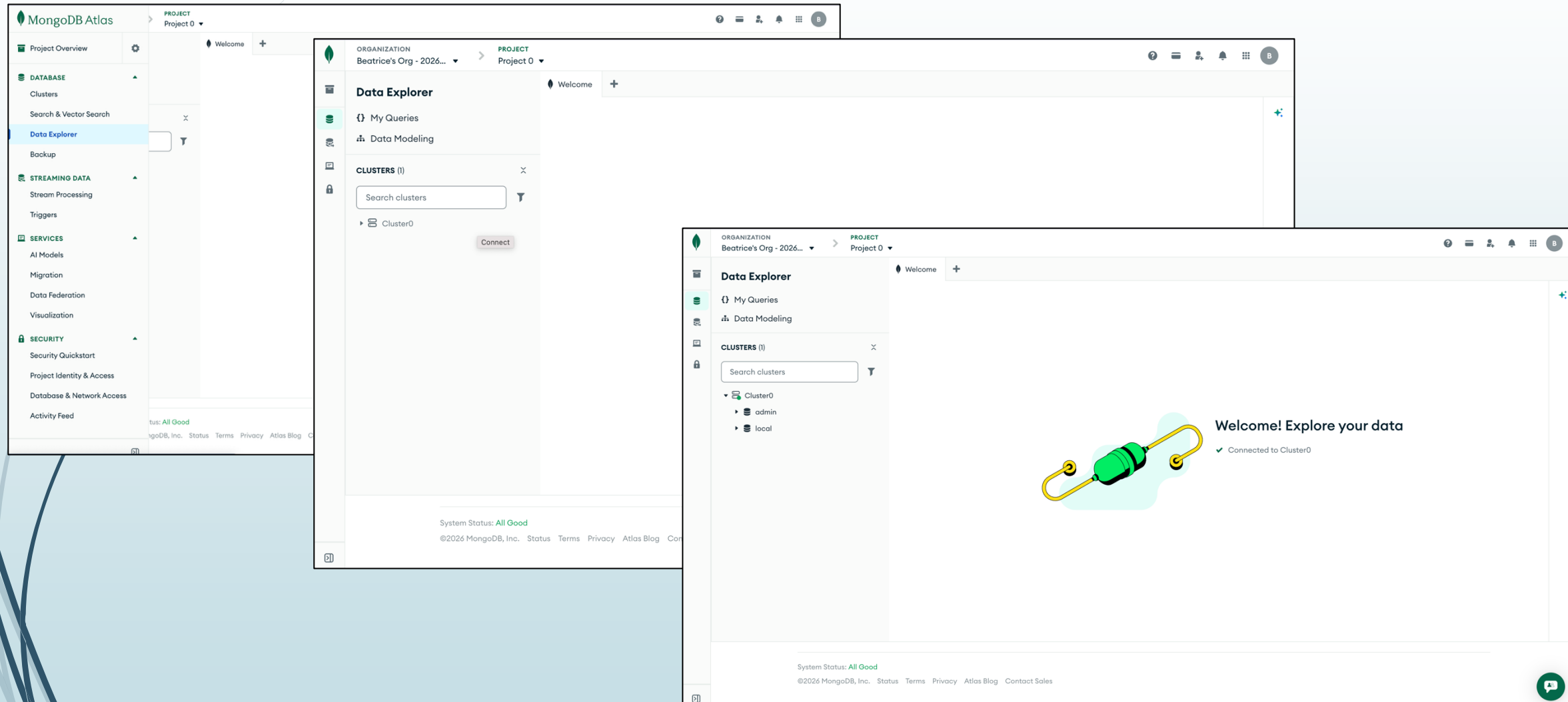
- Interfaccia grafica alternativa all'uso della shell mongosh
- Rappresentazione visuale dei database e delle collezioni
- Visualizza le performance di query ed indici



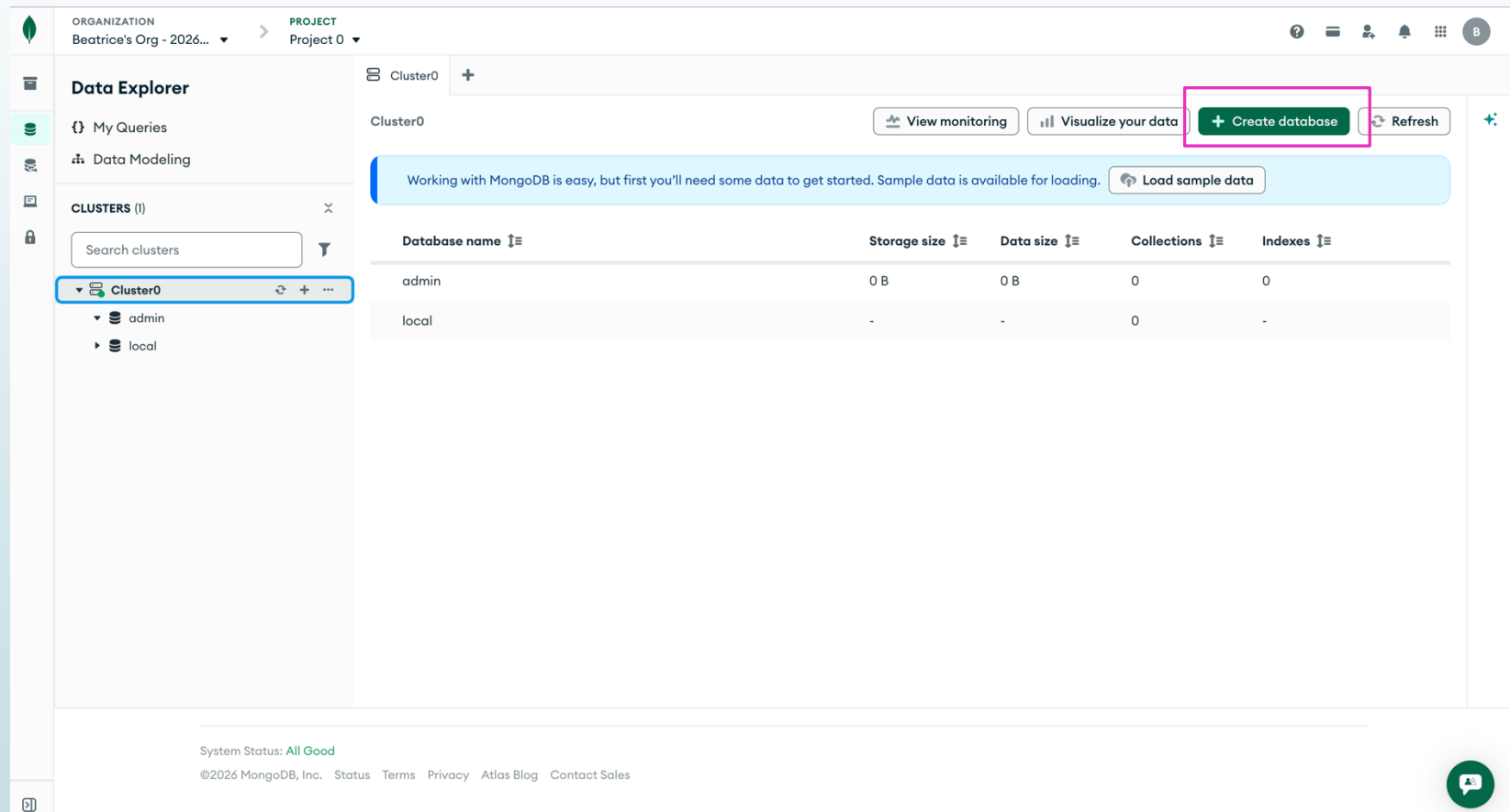
# MongoDB Compass: Connessione



# MongoDB Compass: Consultazione DB



# Creazione del DB

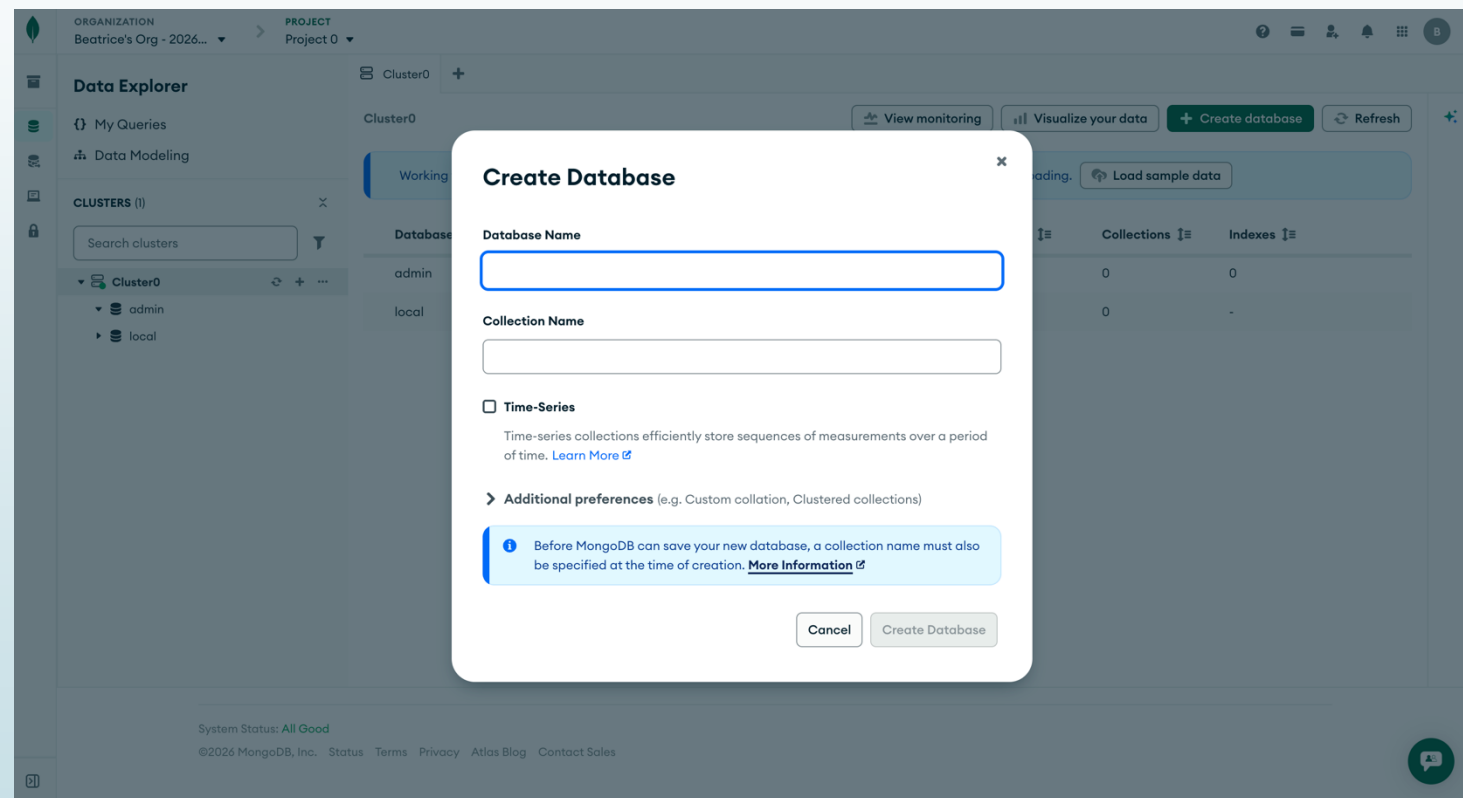


The screenshot displays the MongoDB Atlas Data Explorer interface. On the left sidebar, the 'Data Explorer' section is active, showing a tree view of clusters. 'Cluster0' is selected and highlighted with a blue border. The main panel shows the details for 'Cluster0'. At the top of this panel, there are buttons for 'View monitoring', 'Visualize your data', and a green button labeled '+ Create database', which is highlighted with a pink rectangle. To the right of the 'Create database' button is a 'Refresh' button. Below these buttons is a light blue banner with the text: 'Working with MongoDB is easy, but first you'll need some data to get started. Sample data is available for loading.' and a 'Load sample data' button. Below the banner is a table with the following columns: 'Database name', 'Storage size', 'Data size', 'Collections', and 'Indexes'. The table contains two rows: 'admin' and 'local'.

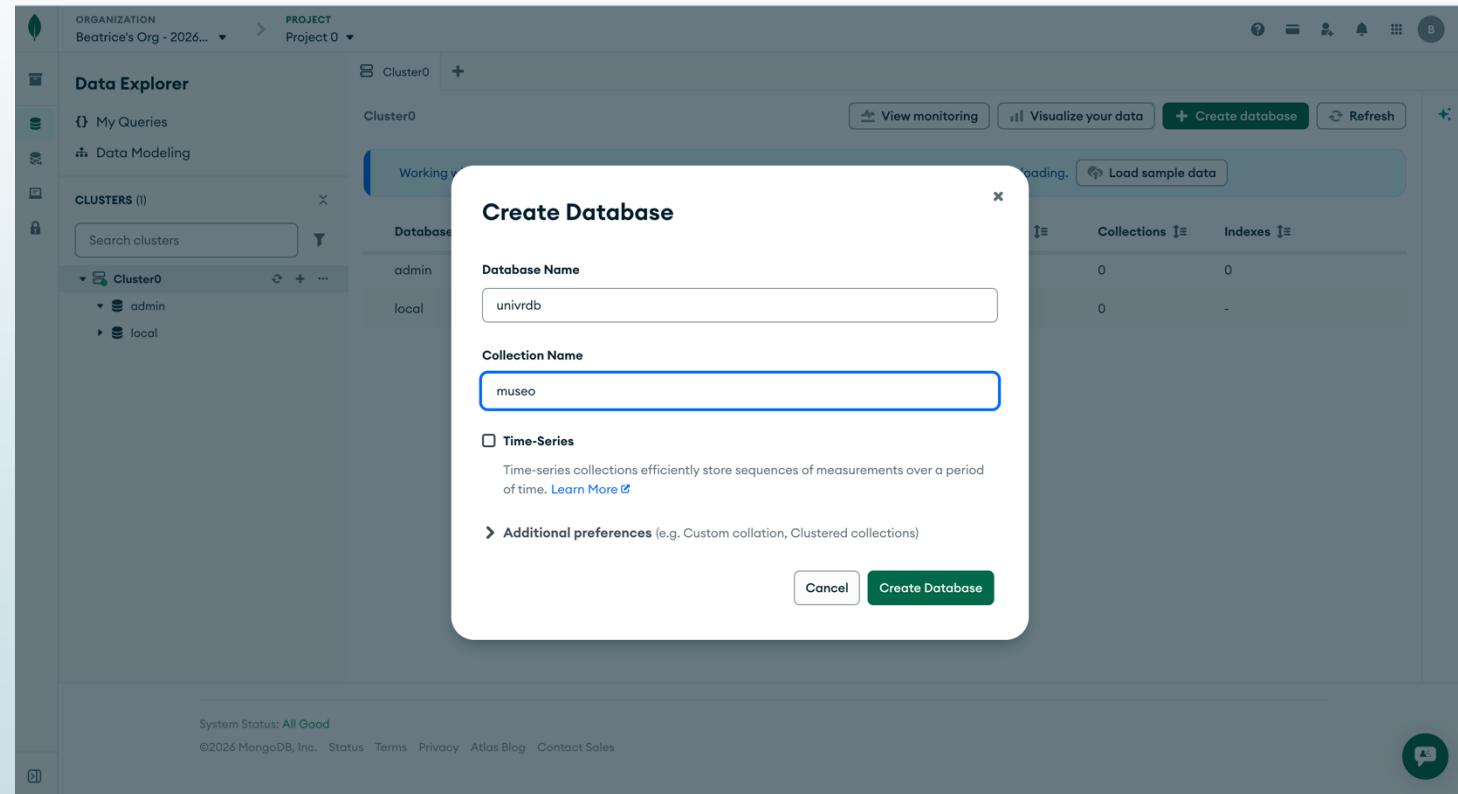
| Database name | Storage size | Data size | Collections | Indexes |
|---------------|--------------|-----------|-------------|---------|
| admin         | 0 B          | 0 B       | 0           | 0       |
| local         | -            | -         | 0           | -       |

System Status: All Good  
©2026 MongoDB, Inc. Status Terms Privacy Atlas Blog Contact Sales

# Creazione del DB



# Creazione del DB



# Creazione del DB

The screenshot displays the MongoDB Atlas Data Explorer interface. The top navigation bar shows the organization 'Beatrice's Org - 2026...' and the project 'Project 0'. The left sidebar contains the 'Data Explorer' menu with options for 'My Queries' and 'Data Modeling'. Below this, the 'CLUSTERS (1)' section shows a search bar and a tree view of the cluster structure: 'Cluster0' (expanded) containing 'admin', 'local', 'univddb', and 'museo'. The main panel shows the 'Cluster0' overview with buttons for 'View monitoring', 'Visualize your data', 'Create database', and 'Refresh'. A table lists the databases within the cluster:

| Database name | Storage size | Data size | Collections | Indexes |
|---------------|--------------|-----------|-------------|---------|
| admin         | 0 B          | 0 B       | 0           | 0       |
| local         | -            | -         | 0           | -       |
| univddb       | 4.10 kB      | 0 B       | 1           | 1       |

At the bottom, the system status is 'All Good', and the footer includes copyright information for MongoDB, Inc. and links to Status, Terms, Privacy, Atlas Blog, and Contact Sales.

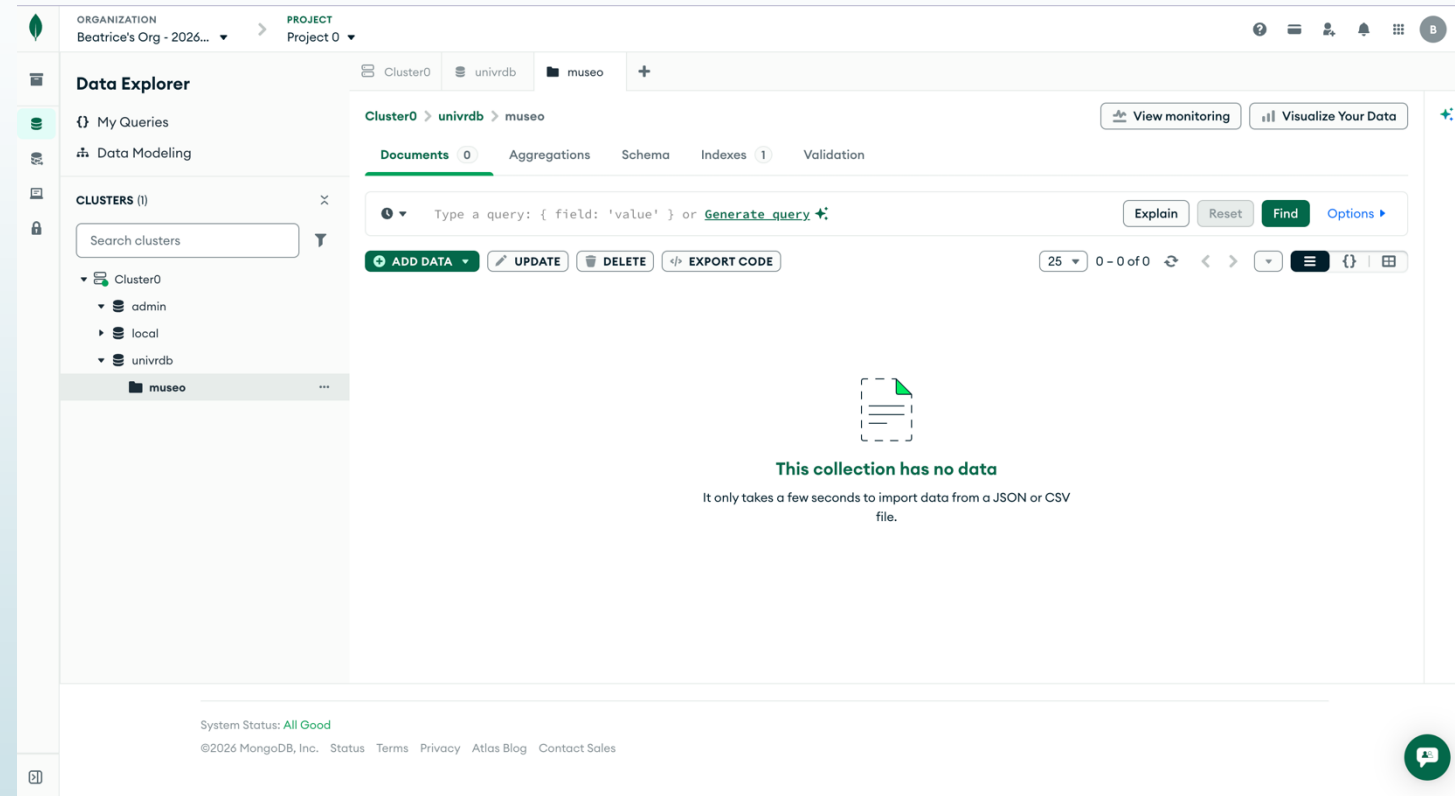
# Creazione del DB

The screenshot displays the MongoDB Atlas Data Explorer interface. The top navigation bar shows the organization 'Beatrice's Org - 2026...' and the project 'Project 0'. The left sidebar contains the 'Data Explorer' menu with options for 'My Queries', 'Data Modeling', and 'CLUSTERS (1)'. The 'CLUSTERS (1)' section shows a tree view with 'Cluster0' expanded, containing 'admin', 'local', and 'univddb' (highlighted with a blue border). Below 'univddb' is a folder named 'museo'. The main panel shows the 'Cluster0 > univddb' view. It includes buttons for 'View monitoring', 'Visualize your data', 'Create collection', and 'Refresh'. A table lists the collections in the database:

| Collection name | Properties | Storage size | Data size | Documents | Avg. document size | Indexes | Total index size |
|-----------------|------------|--------------|-----------|-----------|--------------------|---------|------------------|
| museo           | -          | 4.10 kB      | 0 B       | 0         | 0 B                | 1       | 4.10 kB          |

At the bottom of the interface, the system status is 'All Good', and the footer includes the copyright notice '©2026 MongoDB, Inc.' and links for 'Status', 'Terms', 'Privacy', 'Atlas Blog', and 'Contact Sales'.

# Creazione del DB



Una volta creato un database ed una collezione è possibile importare i dati (documenti) che fanno parte della connessione tramite file CSV oppure JSON.

# Creazione del DB

The screenshot displays the MongoDB Atlas Data Explorer interface. On the left, the 'Data Explorer' sidebar shows the 'CLUSTERS (1)' section with a tree view containing 'Cluster0', 'admin', 'local', 'univddb', and 'museo'. The main panel shows the 'Cluster0 > univddb > museo' path. Below this, tabs for 'Documents', 'Aggregations', 'Schema', 'Indexes', and 'Validation' are visible. The 'Documents' tab is active, showing a query input field with the placeholder text 'Type a query: { field: 'value' } or [Generate query](#)'. To the right of the input field are buttons for 'Explain', 'Reset', 'Find', and 'Options'. A red box highlights the input field, and a blue arrow points to the 'Options' button. Below the query input, there are buttons for 'ADD DATA', 'UPDATE', 'DELETE', and 'EXPORT CODE'. At the bottom of the main panel, a message states 'This collection has no data' with a subtext 'It only takes a few seconds to import data from a JSON or CSV file.' The footer of the interface shows the system status as 'All Good' and copyright information for MongoDB, Inc.

Selezionata una collezione è possibile vederne un campione oppure interrogarla tramite Query textbox.

# Creazione del DB

ORGANIZATION: Beatrice's Org - 2026... PROJECT: Project 0

**Data Explorer**

- My Queries
- Data Modeling

**CLUSTERS (1)**

Search clusters

Cluster0

Cluster0

View monitoring Visualize your data **Create database** Refresh

| Database name | Storage size | Data size | Collections | Indexes |
|---------------|--------------|-----------|-------------|---------|
| admin         | 0 B          | 0 B       | 0           | 0       |
| local         | -            | -         | 0           | -       |
| univddb       | 4.10 kB      | 0 B       | 1           | 1       |

- View performance metrics
- View cluster overview
- View monitoring
- View query insights
- Disconnect
- Connect via ...

System Status: All Good

©2026 MongoDB, Inc. Status Terms Privacy Atlas Blog Contact Sales

# Creazione del DB

Cluster

ORGANIZATION Beatrice's Org - 2026... PROJECT Project 0 CLUSTER Cluster0

Overview

Real Time

Metrics

Query Diagnostics

Online Archive

Command Line Tools

Infrastructure as Code

SHORTCUTS

Search & Vector Search

Data Explorer

Back to Clusters

Overview

Cluster0

VERSION 8.0.21 REGION AWS Frankfurt (eu-central-1) CLUSTER TIER Free

Connect Configuration ...

Load Sample Dataset

Terminate

ADD TAG

TAGS

Use tags to efficiently label and categorize your clusters. Any tags you apply will display here.

Learn more about tagging.

REGION Frankfurt (eu-central-1)

ac-Infz... shard-00-00.7vninf... SECONDARY

ac-Infz... shard-00-01.7vninf... SECONDARY

ac-Infz... shard-00-02.7vninf... PRIMARY

This is a Free Tier Cluster

If you need a database that's better for high-performance production applications, upgrade to a dedicated cluster.

Upgrade

Operations R: 0 W: 0.03/s

Last 6 Hours

Logical Size 0.0 B

512.0 MB max

0.0 B

Last 30 Days

Connections 4

500 max

Last 6 Hours

System Status: All Good

©2026 MongoDB, Inc. Status Terms Privacy Atlas Blog Contact Sales

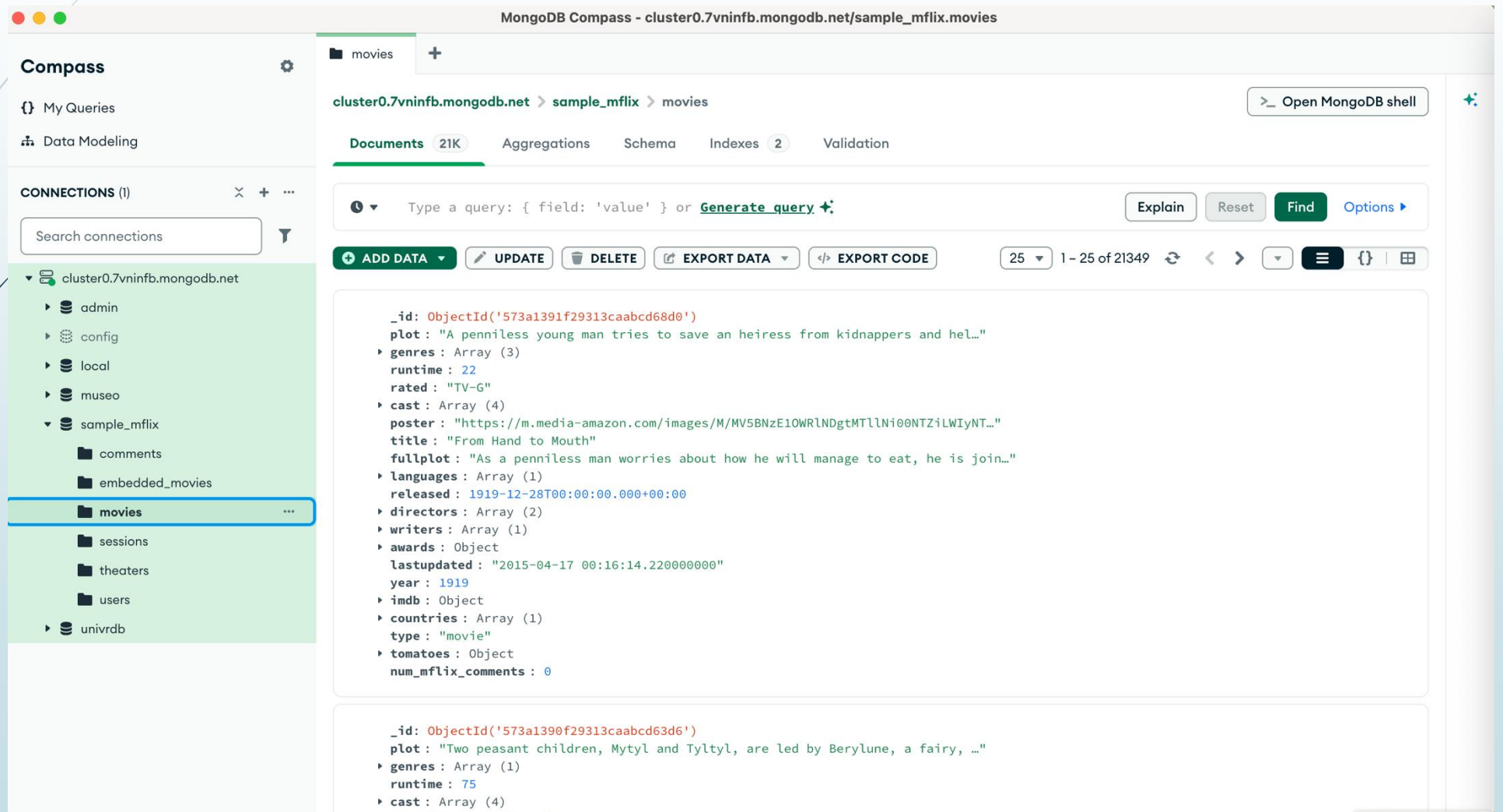
# Creazione del DB

The screenshot displays the MongoDB Atlas 'Cluster Overview' page. A modal dialog titled 'Load sample data' is open in the center. The dialog contains the following elements:

- Header:** 'Load sample data' with a close button (X).
- Text:** 'Based on your data types and use case, Atlas found these recommended sample datasets (3) for you to explore.'
- Search Bar:** Labeled 'Select sample datasets', containing the text 'sample\_mflix' and a clear button (X).
- Dataset List:**
  - ☐ Select All
  - ☒ **sample\_mflix**  
Movies and theaters, includes vector embeddings
  - ☐ **sample\_supplies**  
Customer info and sales
  - ☐ **sample\_airbnb**  
Vacation listings and reviews
  - ☐ **sample\_analytics**  
Financial performance

The background interface shows the 'Overview' tab for 'Cluster0' in the 'AWS Frankfurt (eu-central-1)' region. It includes a sidebar with navigation options like 'Overview', 'Real Time', 'Metrics', and 'Query Diagnostics'. The main area displays 'TAGS' and 'REGION' information. At the bottom, there is a 'System Status: All Good' message and a footer with copyright information for MongoDB, Inc. (©2026).

# Creazione del DB



MongoDB Compass - cluster0.7vninfb.mongodb.net/sample\_mflix.movies

Compass

My Queries

Data Modeling

CONNECTIONS (1)

Search connections

- cluster0.7vninfb.mongodb.net
  - admin
  - config
  - local
  - museo
  - sample\_mflix
    - comments
    - embedded\_movies
    - movies**
    - sessions
    - theaters
    - users
  - univrd

cluster0.7vninfb.mongodb.net > sample\_mflix > movies

Documents 21K Aggregations Schema Indexes 2 Validation

Type a query: { field: 'value' } or [Generate query](#)

Explain Reset Find Options

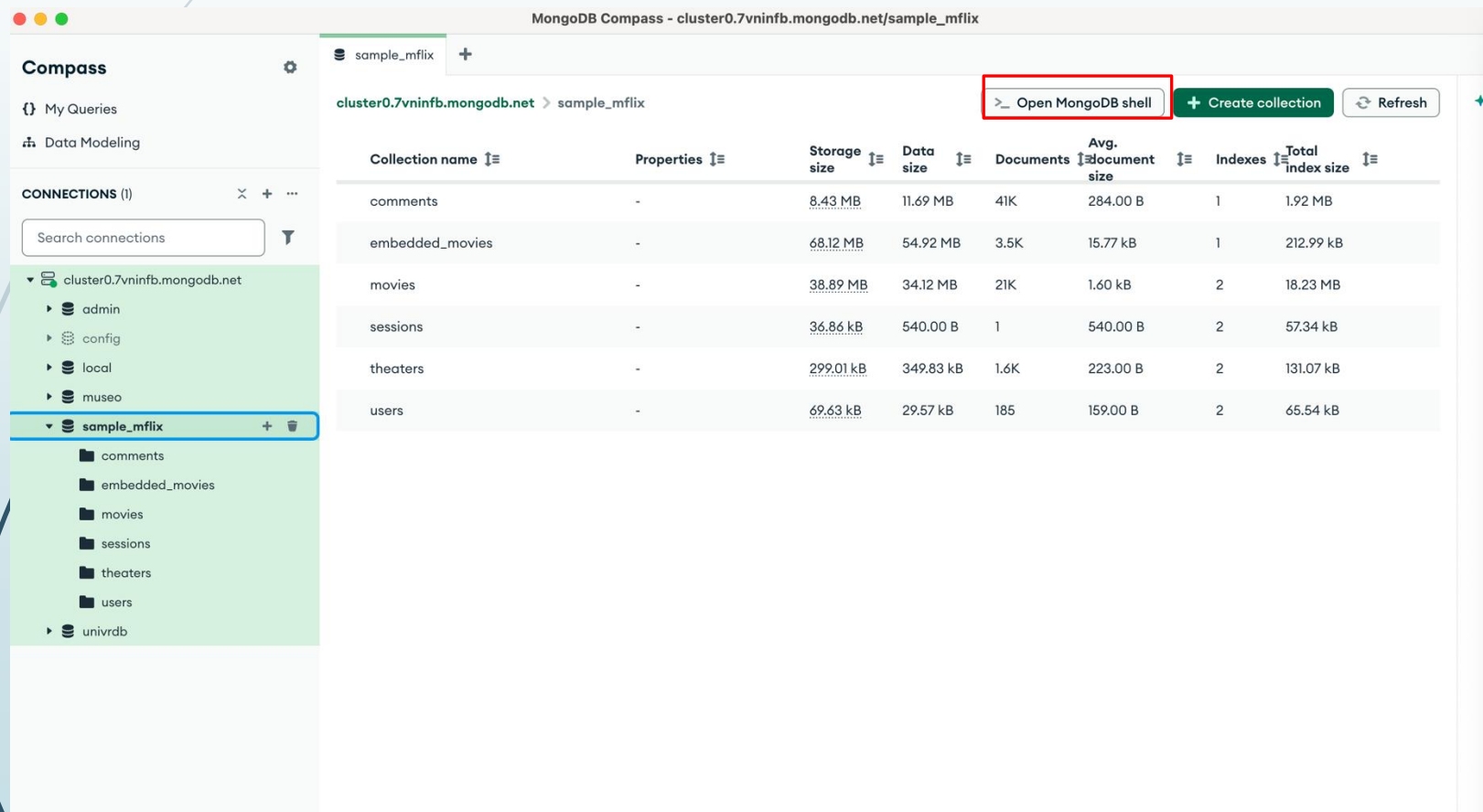
ADD DATA UPDATE DELETE EXPORT DATA EXPORT CODE

25 1 - 25 of 21349

```
{
  "_id": ObjectId('573a1391f29313caabcd68d0'),
  "plot": "A penniless young man tries to save an heiress from kidnappers and hel...",
  "genres": Array (3)
  runtime: 22
  rated: "TV-G"
  cast: Array (4)
  poster: "https://m.media-amazon.com/images/M/MV5BNzE1OWRlNDgtMTllNi00NTZiLWIyNT..."
  title: "From Hand to Mouth"
  fullplot: "As a penniless man worries about how he will manage to eat, he is join..."
  languages: Array (1)
  released: 1919-12-28T00:00:00.000+00:00
  directors: Array (2)
  writers: Array (1)
  awards: Object
  lastupdated: "2015-04-17 00:16:14.220000000"
  year: 1919
  imdb: Object
  countries: Array (1)
  type: "movie"
  tomatoes: Object
  num_mflix_comments: 0
}
```

```
{
  "_id": ObjectId('573a1390f29313caabcd63d6'),
  "plot": "Two peasant children, Mytyl and Tytyl, are led by Berylune, a fairy, ..."
  genres: Array (1)
  runtime: 75
  cast: Array (4)
}
```

# MongoDB Compass: Creazione Database

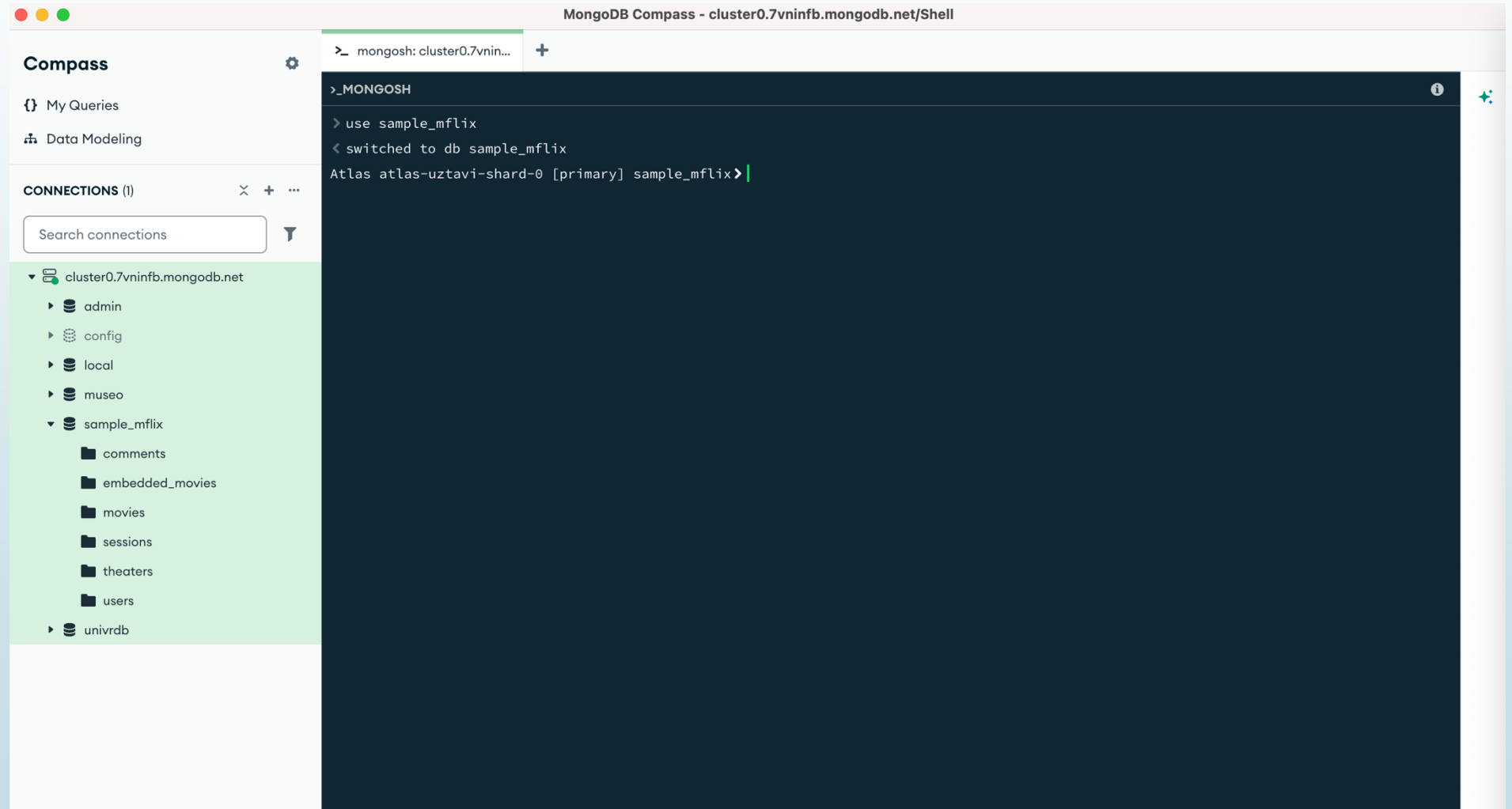


The screenshot shows the MongoDB Compass interface. On the left, the 'CONNECTIONS' panel lists several databases, with 'sample\_mflix' selected. The main panel displays the 'sample\_mflix' database with a table of collections. A red box highlights the 'Open MongoDB shell' button in the top right corner of the main panel. A blue arrow points from the text on the right towards this button.

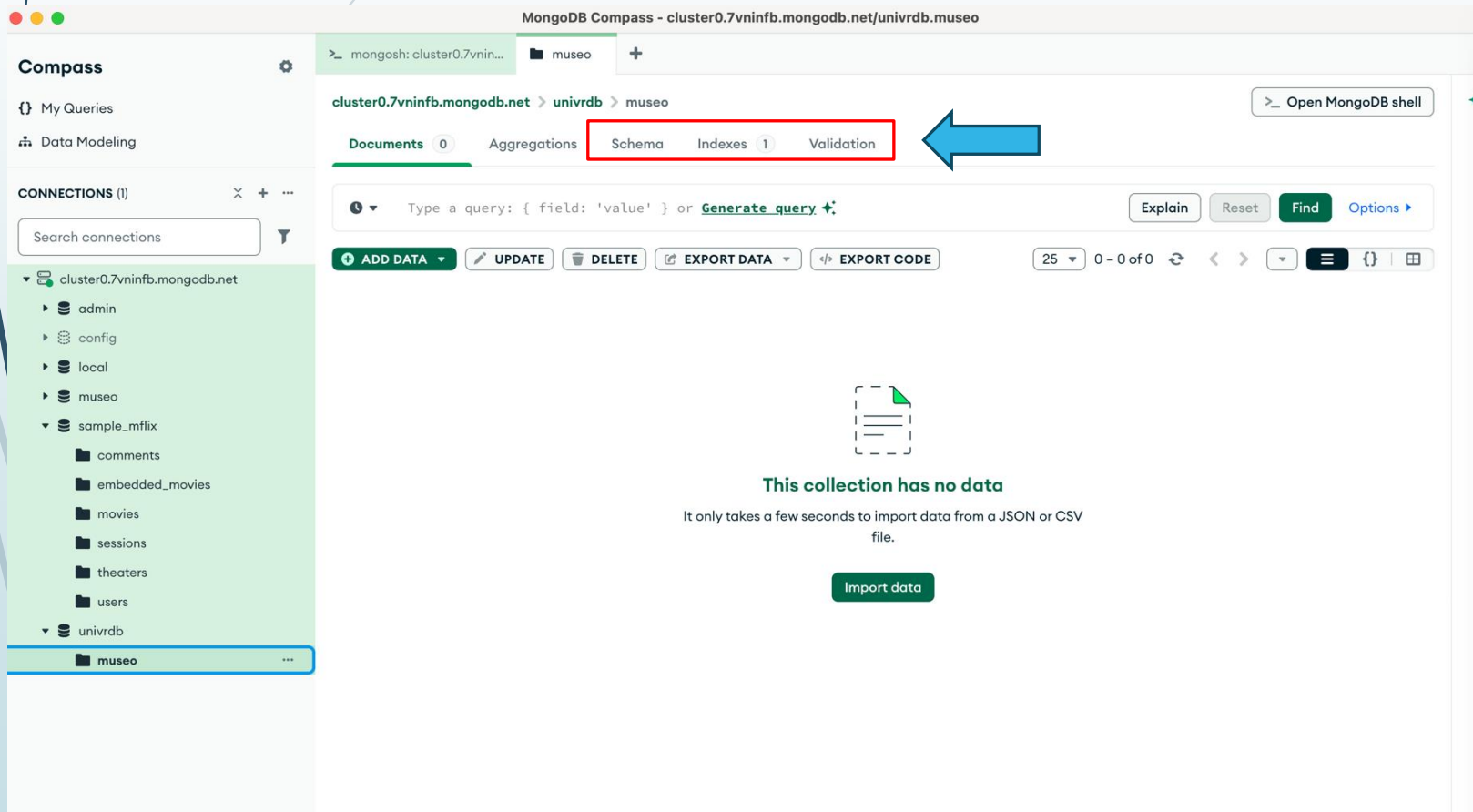
| Collection name | Properties | Storage size | Data size | Documents | Avg. document size | Indexes | Total index size |
|-----------------|------------|--------------|-----------|-----------|--------------------|---------|------------------|
| comments        | -          | 8.43 MB      | 11.69 MB  | 41K       | 284.00 B           | 1       | 1.92 MB          |
| embedded_movies | -          | 68.12 MB     | 54.92 MB  | 3.5K      | 15.77 kB           | 1       | 212.99 kB        |
| movies          | -          | 38.89 MB     | 34.12 MB  | 21K       | 1.60 kB            | 2       | 18.23 MB         |
| sessions        | -          | 36.86 kB     | 540.00 B  | 1         | 540.00 B           | 2       | 57.34 kB         |
| theaters        | -          | 299.01 kB    | 349.83 kB | 1.6K      | 223.00 B           | 2       | 131.07 kB        |
| users           | -          | 69.63 kB     | 29.57 kB  | 185       | 159.00 B           | 2       | 65.54 kB         |

Usare la shell MongoDB integrata in Compass.

# MongoDB Compass: utilizzo della shell



# Analisi della collezione



- Vedere lo schema dei documenti della collezione.
- Consultare gli indici della collezione.
- Validation rules: forzare delle regole di struttura dei documenti che devono essere soddisfatte durante gli aggiornamenti e gli inserimenti.

# Indici

MongoDB Compass - cluster0.7vninfb.mongodb.net/sample\_mflix.movies

> mongosh: cluster0.7vninfb.mongodb.net/sample\_mflix.movies

cluster0.7vninfb.mongodb.net > sample\_mflix > movies

Documents 21K Aggregations Schema **Indexes 2** Validation

Create Refresh

VIEWING INDEXES SEARCH INDEXES

| Name & Definition                                | Type      | Size    | Usage                     | Properties | Status |
|--------------------------------------------------|-----------|---------|---------------------------|------------|--------|
| > _id_                                           | REGULAR ⓘ | 1.1 MB  | 3 (since Fri May 01 2026) | UNIQUE ⓘ   | READY  |
| > cast_text_fullplot_text_genres_text_title_text | TEXT ⓘ    | 17.1 MB | 0 (since Fri May 01 2026) | COMPOUND ⓘ | READY  |

Compass

My Queries

Data Modeling

CONNECTIONS (1)

Search connections

- cluster0.7vninfb.mongodb.net
  - admin
  - config
  - local
  - museo
  - sample\_mflix
    - comments
    - embedded\_movies
    - movies**
    - sessions
    - theaters
    - users
  - univrrdb
    - museo



# Operazioni CRUD

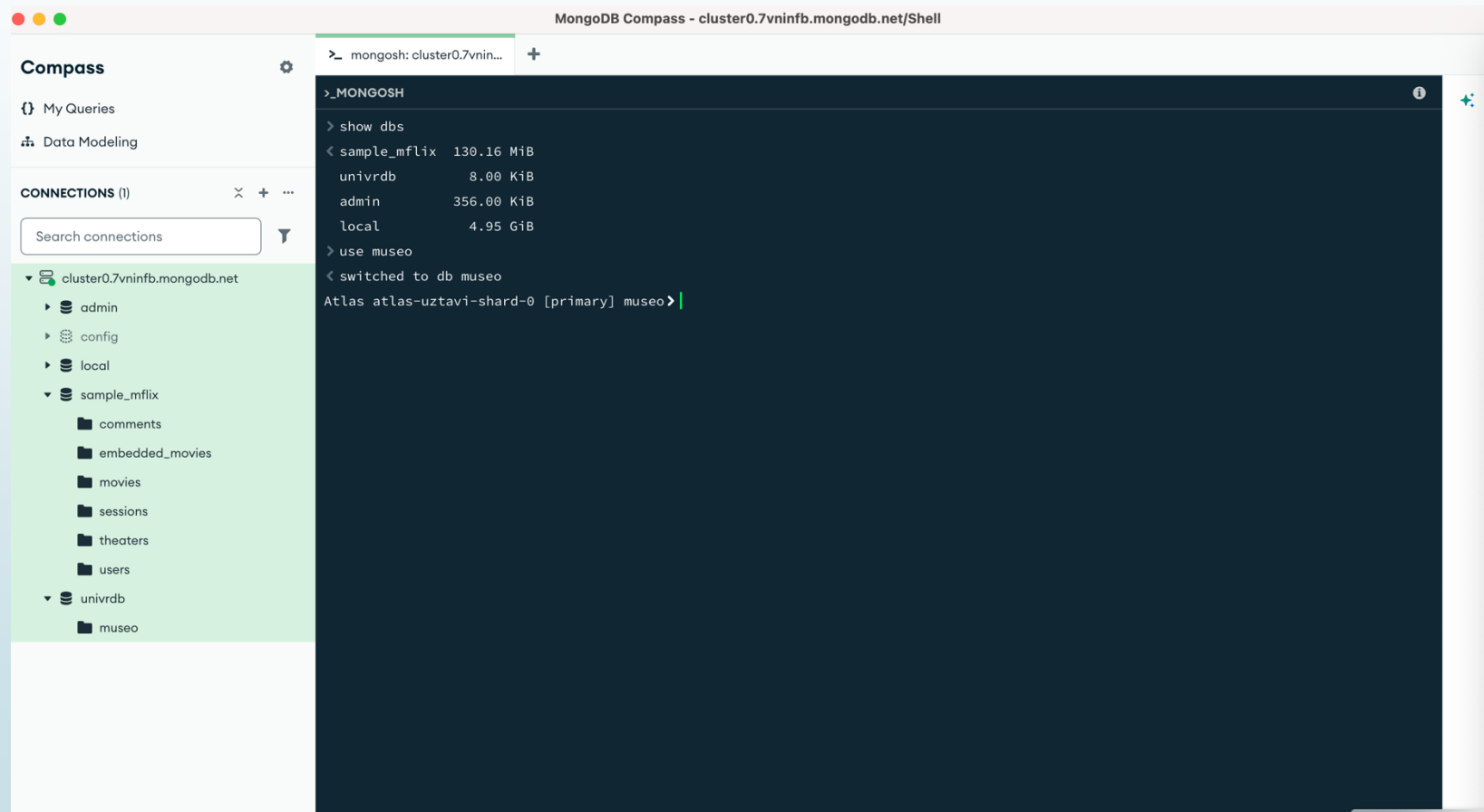
Operazioni tipiche su una base di dati MongoDB

- **Create**: inserire un nuovo documento in un database MongoDB
- **Read**: interrogare uno o più documenti
- **Update**: modificare un documento esistente
- **Delete**: rimuovere un documento esistente

Punto di partenza

- `show dbs`
- `use <dbname>`

# Usare una base di dati specifica





# Create: Inserimento

Riassunto dei Metodi disponibili:

- `db.collection.insertOne()`
- `db.collection.insertMany()`



# Create: Inserimento

insertOne ci permette di inserire un documento in una collezione

```
db.museo.insertOne({ nome: 'Arena',  
  città: 'Verona',  
  indirizzo: 'Piazza Bra',  
  telefono: '04500303',  
  chiusura: 'MAR',  
  prezzo: 20,  
});
```

Nota:

- ▀ `db.museo = db["museo"] = db.collection('museo')`

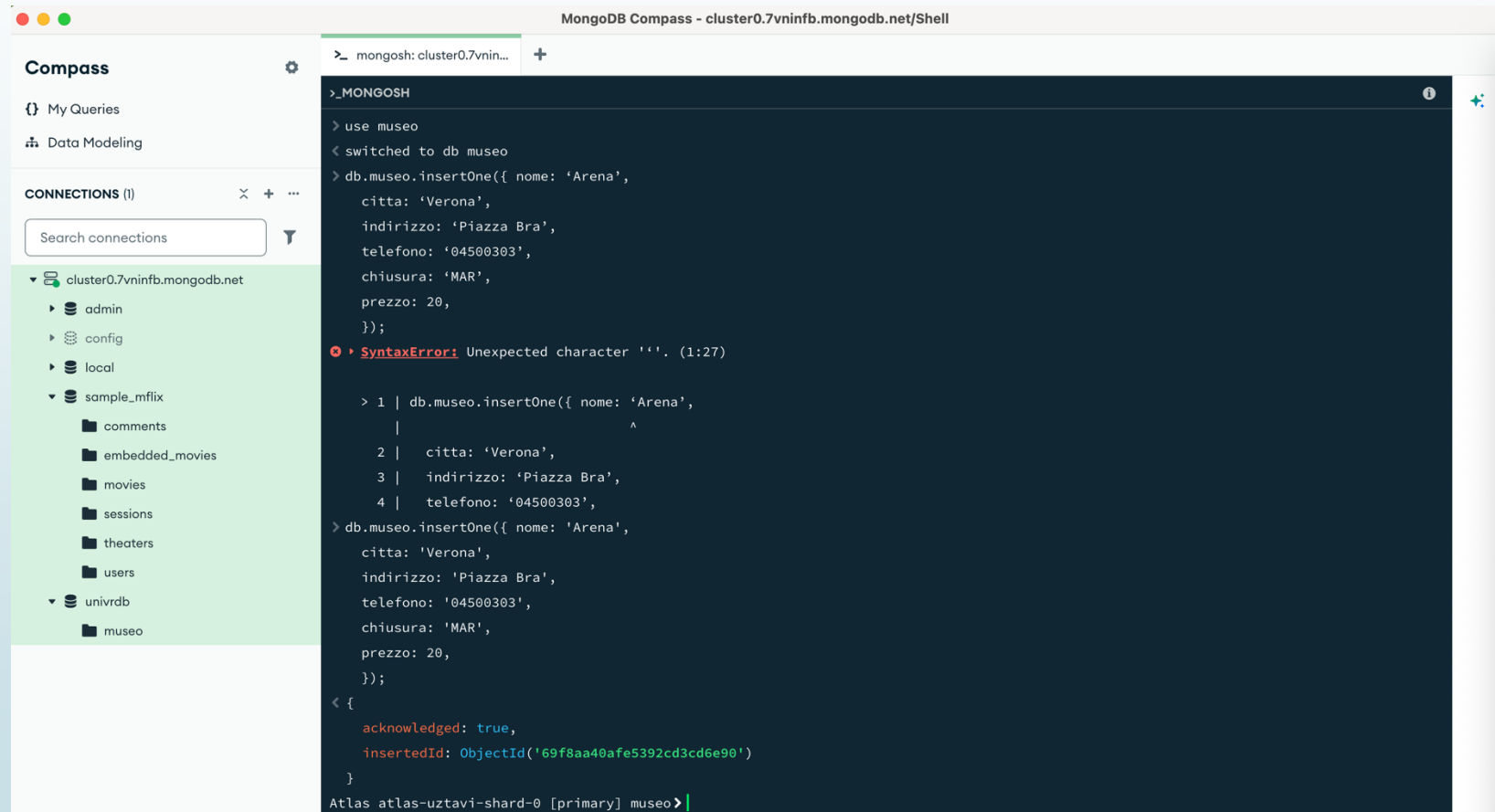
# Create: Inserimento

Riceve in ingresso un oggetto JSON che rappresenta il documento da inserire

Restituisce un oggetto JSON che indica: l'esito dell'operazione e l'objectId dell'oggetto appena inserito.

MongoDB crea automaticamente un identificatore univoco per ogni oggetto (objectId).

Attenzione agli apici!



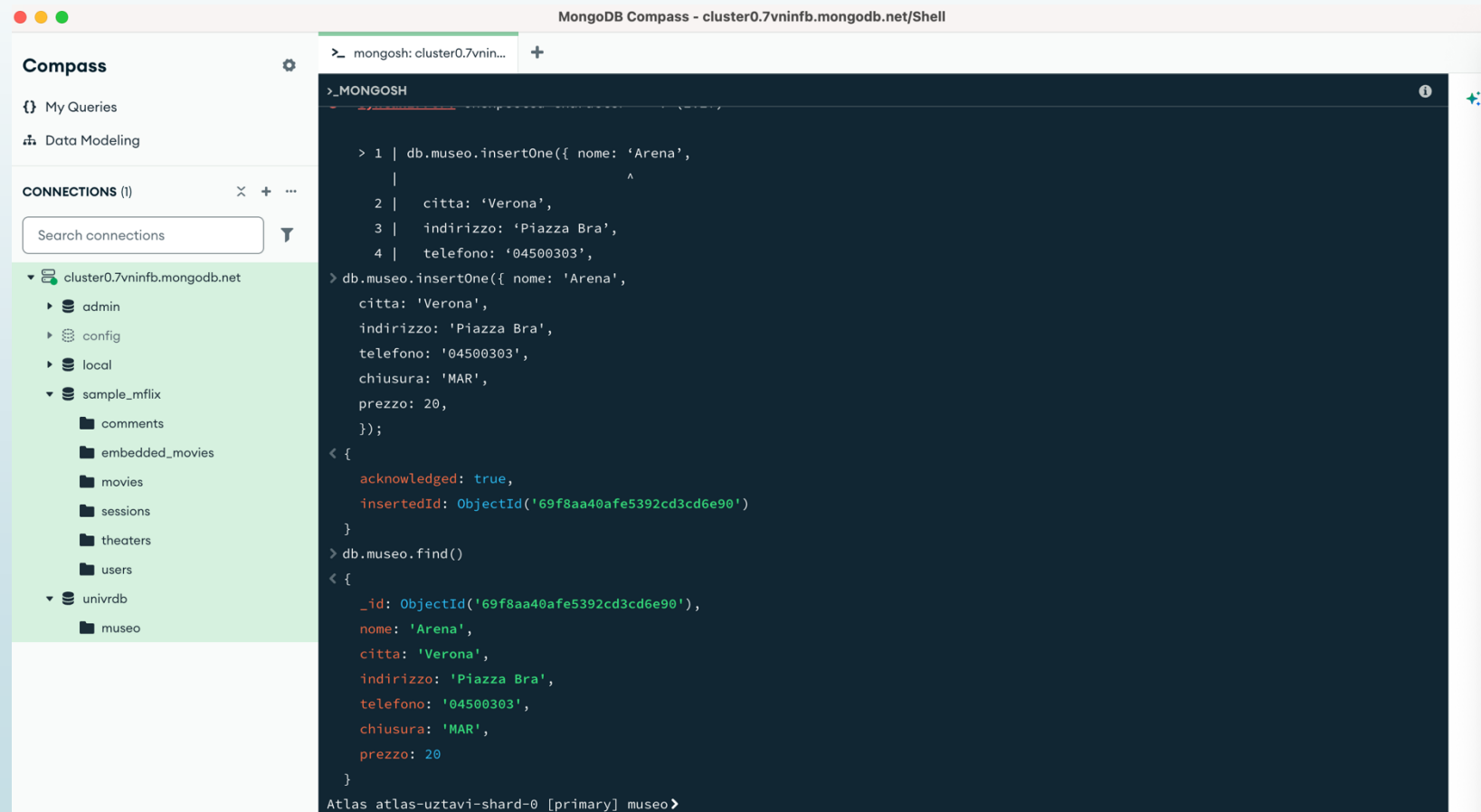
The screenshot shows the MongoDB Compass interface. On the left, the 'COMPASS' sidebar displays the 'My Queries' and 'Data Modeling' tabs. Below these, the 'CONNECTIONS (1)' section shows a list of connections, including 'cluster0.7vninfb.mongodb.net'. The main panel on the right is the 'mongo' shell, showing the following commands and output:

```
> use museo
< switched to db museo
> db.museo.insertOne({ nome: 'Arena',
  citta: 'Verona',
  indirizzo: 'Piazza Bra',
  telefono: '04500303',
  chiusura: 'MAR',
  prezzo: 20,
});
< {
  acknowledged: true,
  insertedId: ObjectId('69f8aa40afe5392cd3cd6e90')
}
```

The output shows a successful insertion with the acknowledgment status and the generated ObjectId. The shell prompt at the bottom indicates the current database is 'museo'.

# Create: Inserimento

db.museo.find()



The screenshot displays the MongoDB Compass application window. On the left sidebar, the 'Connections' panel shows a tree view of the database structure. The 'cluster0.7vninfb.mongodb.net' connection is expanded, revealing a 'museo' database. The main panel on the right is titled 'MongoDB Compass - cluster0.7vninfb.mongodb.net/Shell' and contains a terminal window with the following commands and output:

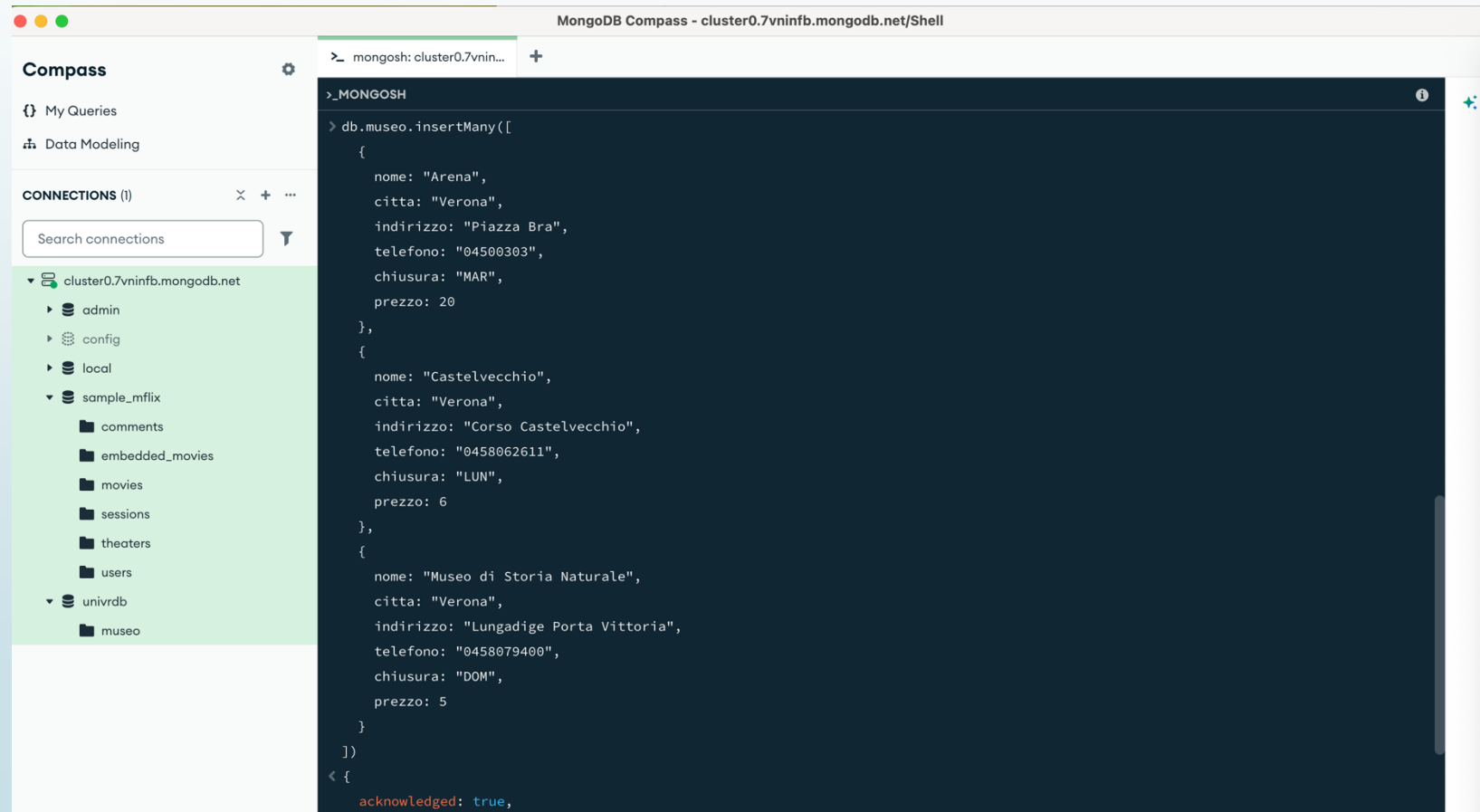
```
> mongosh: cluster0.7vninfb.mongodb.net/Shell
> _MONGOSH
> 1 | db.museo.insertOne({ nome: 'Arena',
    |                       ^
    | citta: 'Verona',
    | indirizzo: 'Piazza Bra',
    | telefono: '04500303',
    |
    | db.museo.insertOne({ nome: 'Arena',
    | citta: 'Verona',
    | indirizzo: 'Piazza Bra',
    | telefono: '04500303',
    | chiusura: 'MAR',
    | prezzo: 20,
    | });
< {
  acknowledged: true,
  insertedId: ObjectId('69f8aa40afe5392cd3cd6e90')
}
> db.museo.find()
< {
  _id: ObjectId('69f8aa40afe5392cd3cd6e90'),
  nome: 'Arena',
  citta: 'Verona',
  indirizzo: 'Piazza Bra',
  telefono: '04500303',
  chiusura: 'MAR',
  prezzo: 20
}
Atlas atlas-uztavi-shard-0 [primary] museo>
```



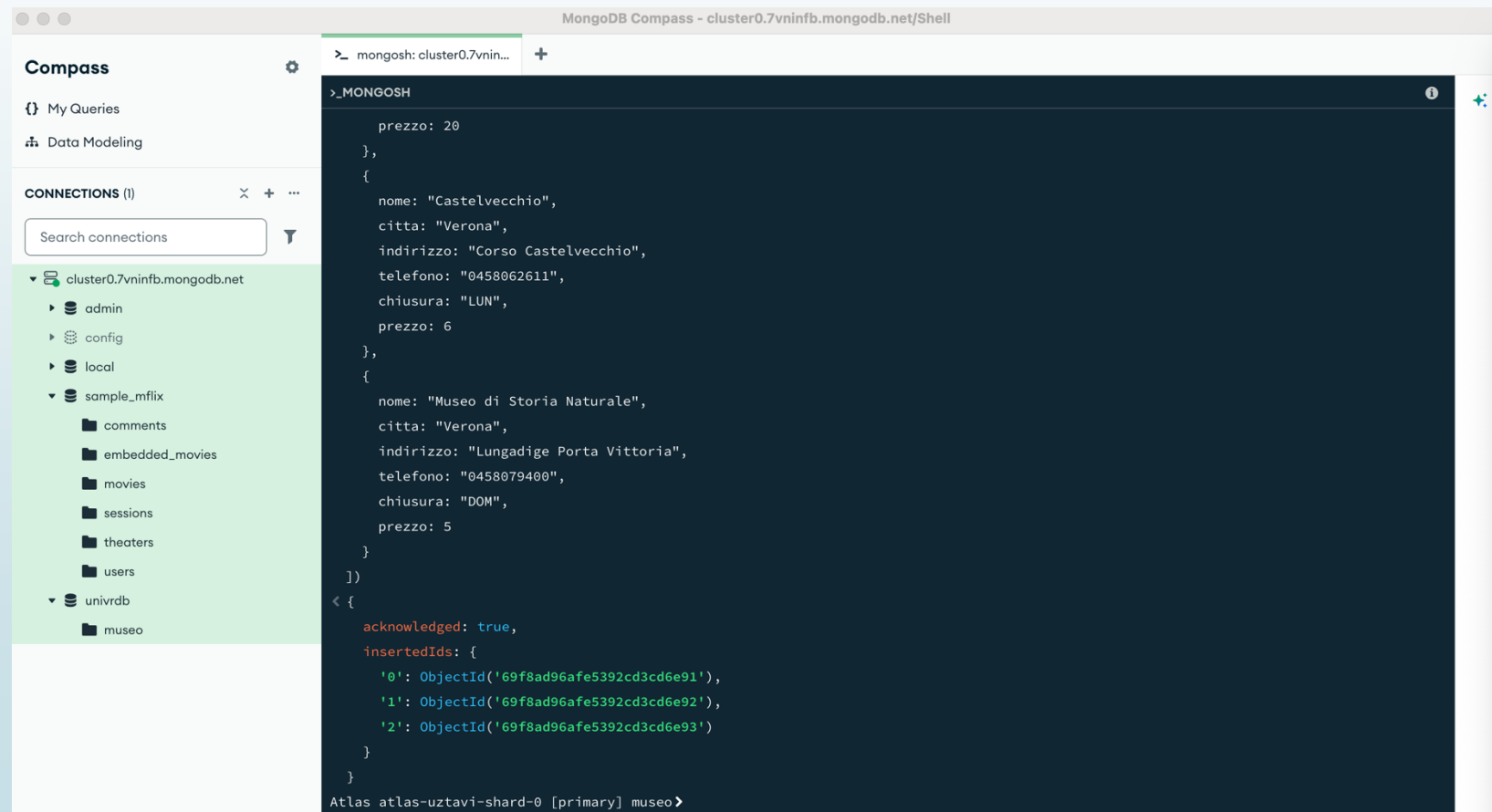
# Create: Inserimento multiplo

```
db.museo.insertMany([
  { nome: "Arena",
    citta: "Verona",
    ..... },
  {nome: "Castelvecchio",
    citta: "Verona",
    ..... },
  {nome: "Museo di Storia Naturale",
    citta: "Verona",
    ... }
])
```

# Inserimento multiplo



# Inserimento multiplo



MongoDB Compass - cluster0.7vninfb.mongodb.net/Shell

Compass

My Queries

Data Modeling

CONNECTIONS (1)

Search connections

cluster0.7vninfb.mongodb.net

- admin
- config
- local
- sample\_mflix
  - comments
  - embedded\_movies
  - movies
  - sessions
  - theaters
  - users
- univrdp
  - museo

```
> mongosh: cluster0.7vninfb.mongodb.net/Shell
>_MONGOSH
> db.museo.find()
< {
  "_id": ObjectId('69f8aa40afe5392cd3cd6e90'),
  "nome": 'Arena',
  "citta": 'Verona',
  "indirizzo": 'Piazza Bra',
  "telefono": '04500303',
  "chiusura": 'MAR',
  "prezzo": 20
}
{
  "_id": ObjectId('69f8ad96afe5392cd3cd6e91'),
  "nome": 'Arena',
  "citta": 'Verona',
  "indirizzo": 'Piazza Bra',
  "telefono": '04500303',
  "chiusura": 'MAR',
  "prezzo": 20
}
{
  "_id": ObjectId('69f8ad96afe5392cd3cd6e92'),
  "nome": 'Castelvecchio',
  "citta": 'Verona',
  "indirizzo": 'Corso Castelvecchio',
  "telefono": '0458062611',
  "chiusura": 'LUN',
  "prezzo": 6
}
-
{
  "_id": ObjectId('69f8ad96afe5392cd3cd6e93'),
  "nome": 'Museo di Storia Naturale',
  "citta": 'Verona',
  "indirizzo": 'Lungadige Porta Vittoria',
  "telefono": '0458079400',
  "chiusura": 'DOM',
  "prezzo": 5
}
}
Atlas atlas-uztavi-shard-0 [primary] museo>
```



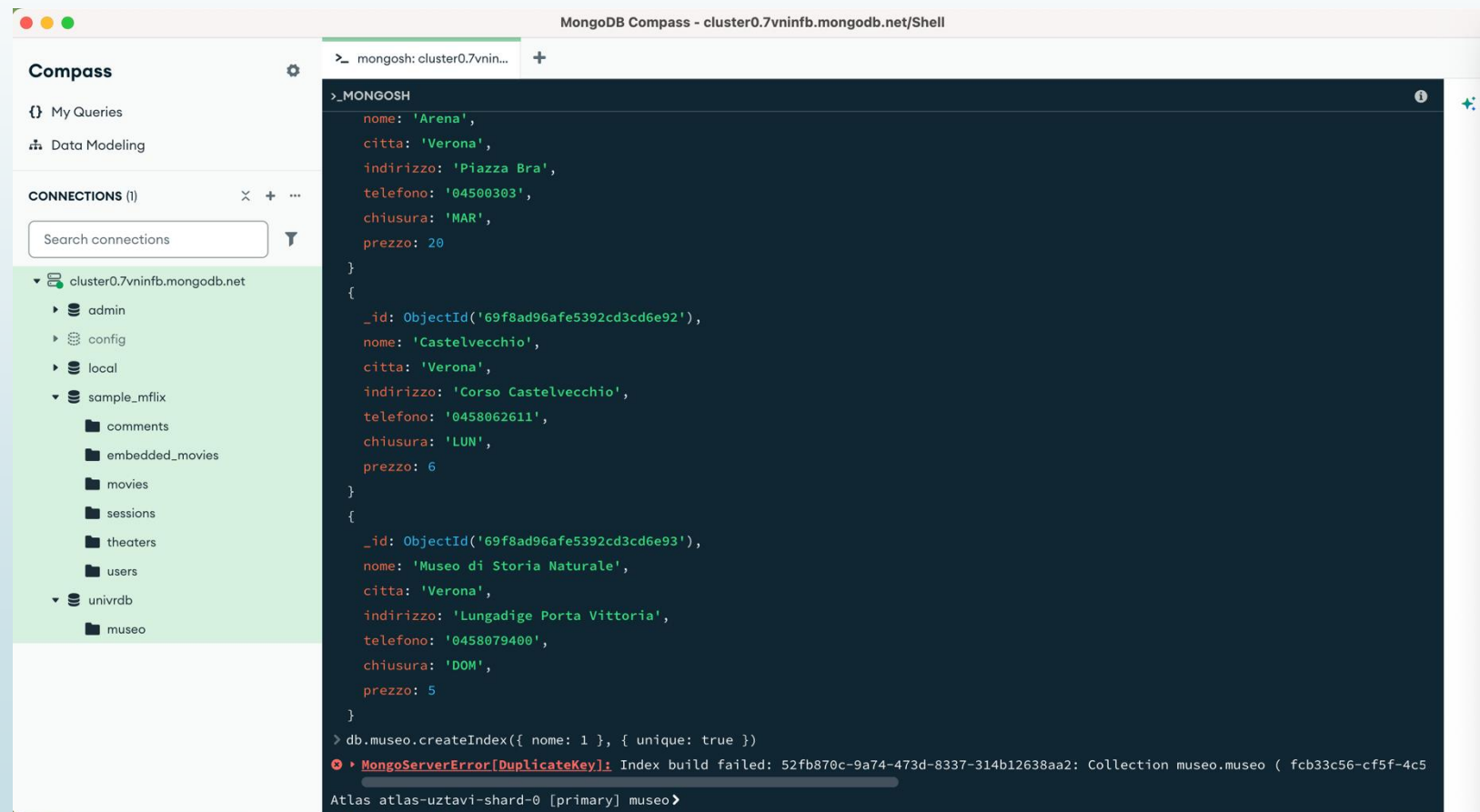
# Inserimento duplicati

```
db.museo.createIndex({ nome: 1 }, { unique: true })
```

Proviamo:

```
db.museo.insertOne({ nome: 'Arena',  
  città: 'Verona',  
  indirizzo: 'Piazza Bra',  
  telefono: '04500303',  
  chiusura: 'MAR',  
  prezzo: 20,  
});
```

# Inserimento duplicati



The screenshot shows the MongoDB Compass application. On the left, the 'Connections' panel lists the connected cluster 'cluster0.7vninfb.mongodb.net' and its databases, including 'sample\_mflix' and 'univrd' with a 'museo' collection. The main panel displays the MongoDB Shell with a JSON document being inserted into the 'museo' collection. The document has a unique '\_id' but a duplicate 'nome' value ('Museo di Storia Naturale') compared to an existing document. This results in a 'MongoServerError[E DuplicateKey]: Index build failed' error.

```
> mongosh: cluster0.7vninfb.mongodb.net/Shell
>_MONGOSH
{
  nome: 'Arena',
  citta: 'Verona',
  indirizzo: 'Piazza Bra',
  telefono: '04500303',
  chiusura: 'MAR',
  prezzo: 20
}
{
  _id: ObjectId('69f8ad96afe5392cd3cd6e92'),
  nome: 'Castelvecchio',
  citta: 'Verona',
  indirizzo: 'Corso Castelvecchio',
  telefono: '0458062611',
  chiusura: 'LUN',
  prezzo: 6
}
{
  _id: ObjectId('69f8ad96afe5392cd3cd6e93'),
  nome: 'Museo di Storia Naturale',
  citta: 'Verona',
  indirizzo: 'Lungadige Porta Vittoria',
  telefono: '0458079400',
  chiusura: 'DOM',
  prezzo: 5
}
> db.museo.createIndex({ nome: 1 }, { unique: true })
MongoServerError[E DuplicateKey]: Index build failed: 52fb870c-9a74-473d-8337-314b12638aa2: Collection museo.museo ( fcb33c56-cf5f-4c5
Atlas atlas-uztavi-shard-0 [primary] museo>
```



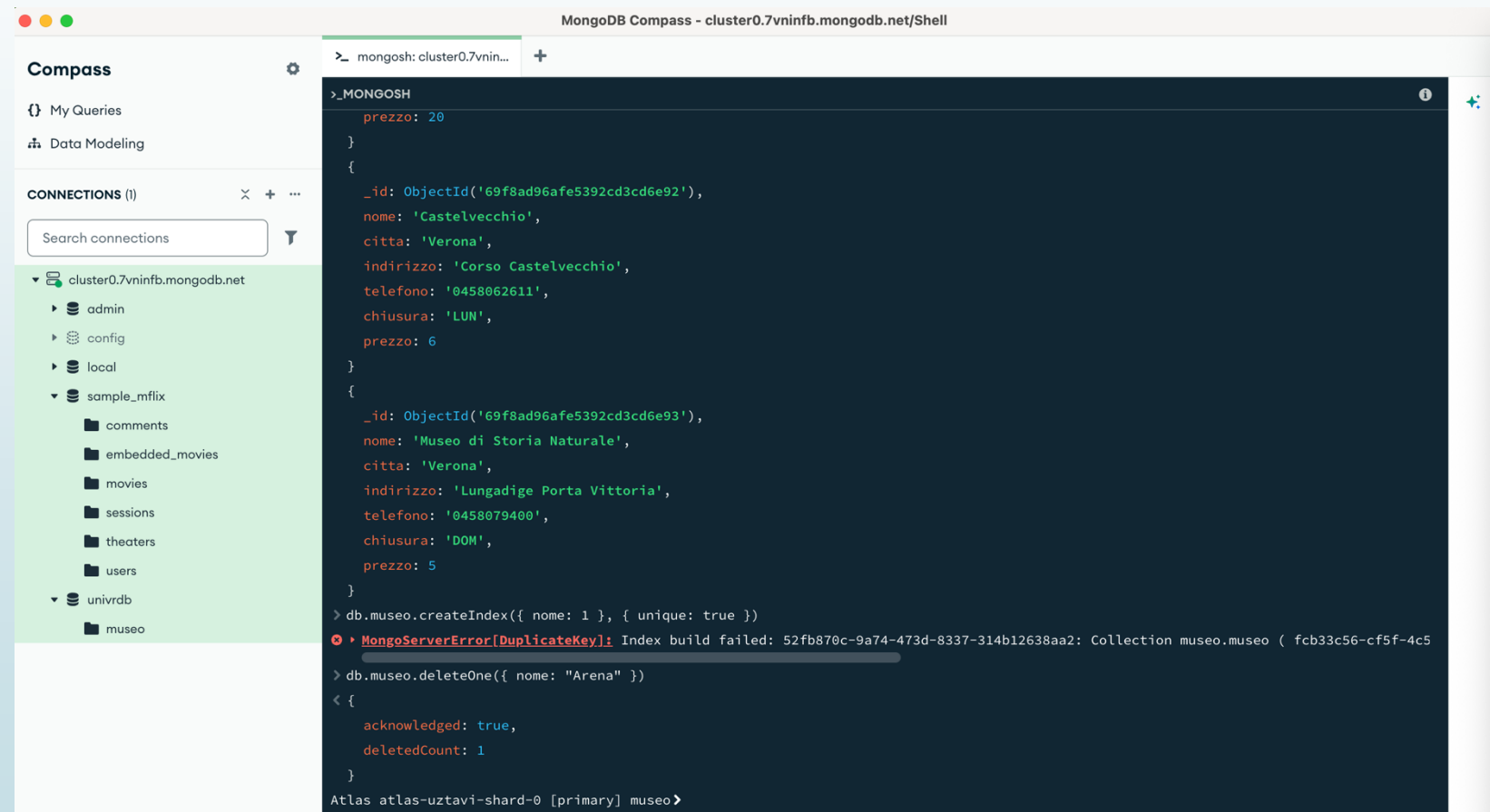
# Inserimento duplicati

In questo caso cancelliamo un elemento con:

`db.colletion.deleteOne()`: Cancella il primo elemento che trova con il riferimento trovato

`db.museo.deleteOne({ nome: "Arena" })`: cancella il primo elemento che trova con il nome Arena.

# Inserimento duplicati

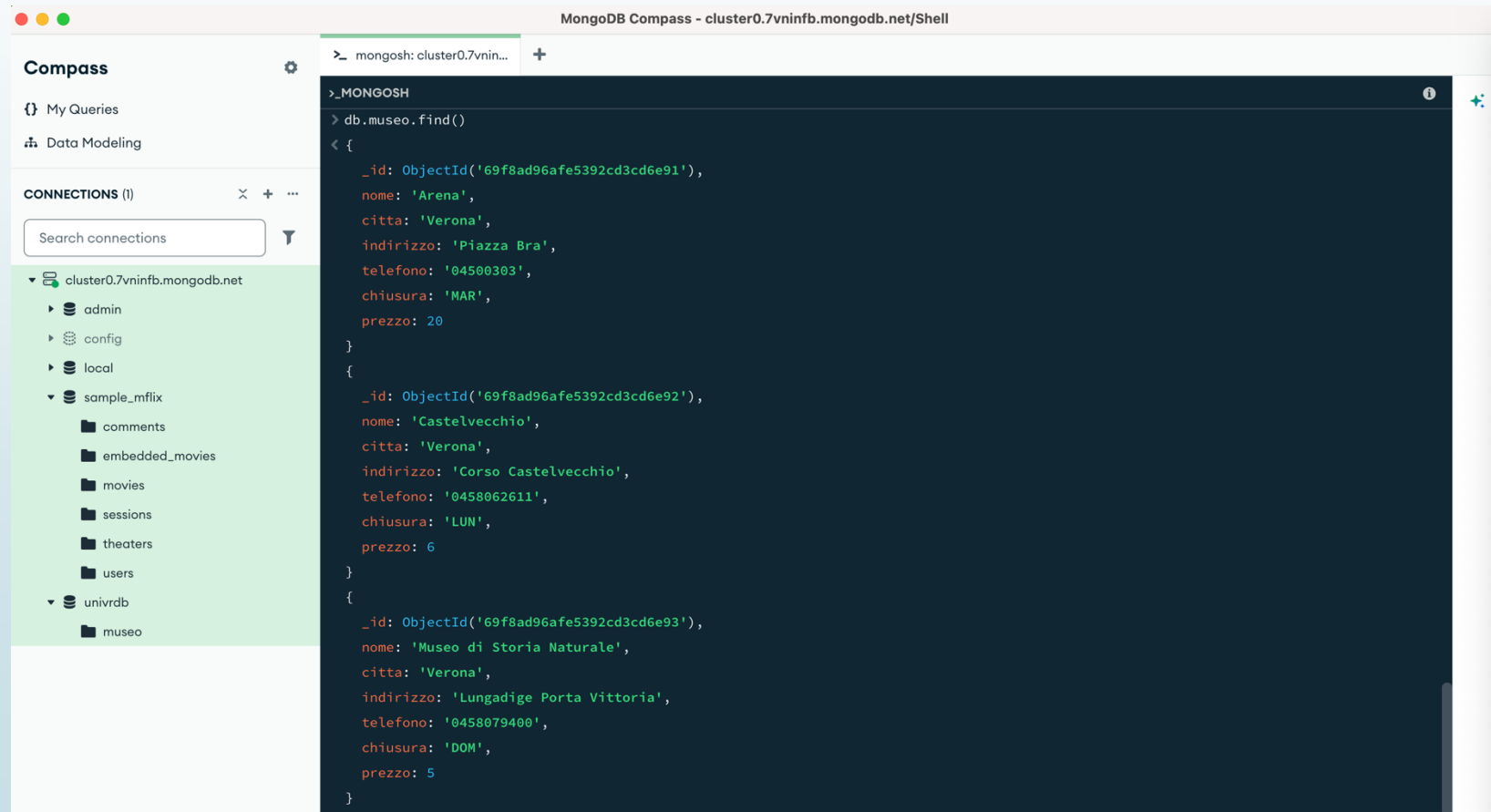


The screenshot shows the MongoDB Compass application window. On the left, the 'Compass' sidebar is visible with sections for 'My Queries', 'Data Modeling', and 'CONNECTIONS (1)'. Under 'CONNECTIONS', a cluster named 'cluster0.7vninfb.mongodb.net' is expanded, showing a tree of databases including 'admin', 'config', 'local', 'sample\_mflix', and 'univrrdb'. The 'univrrdb' database is selected, and the 'museo' collection is visible.

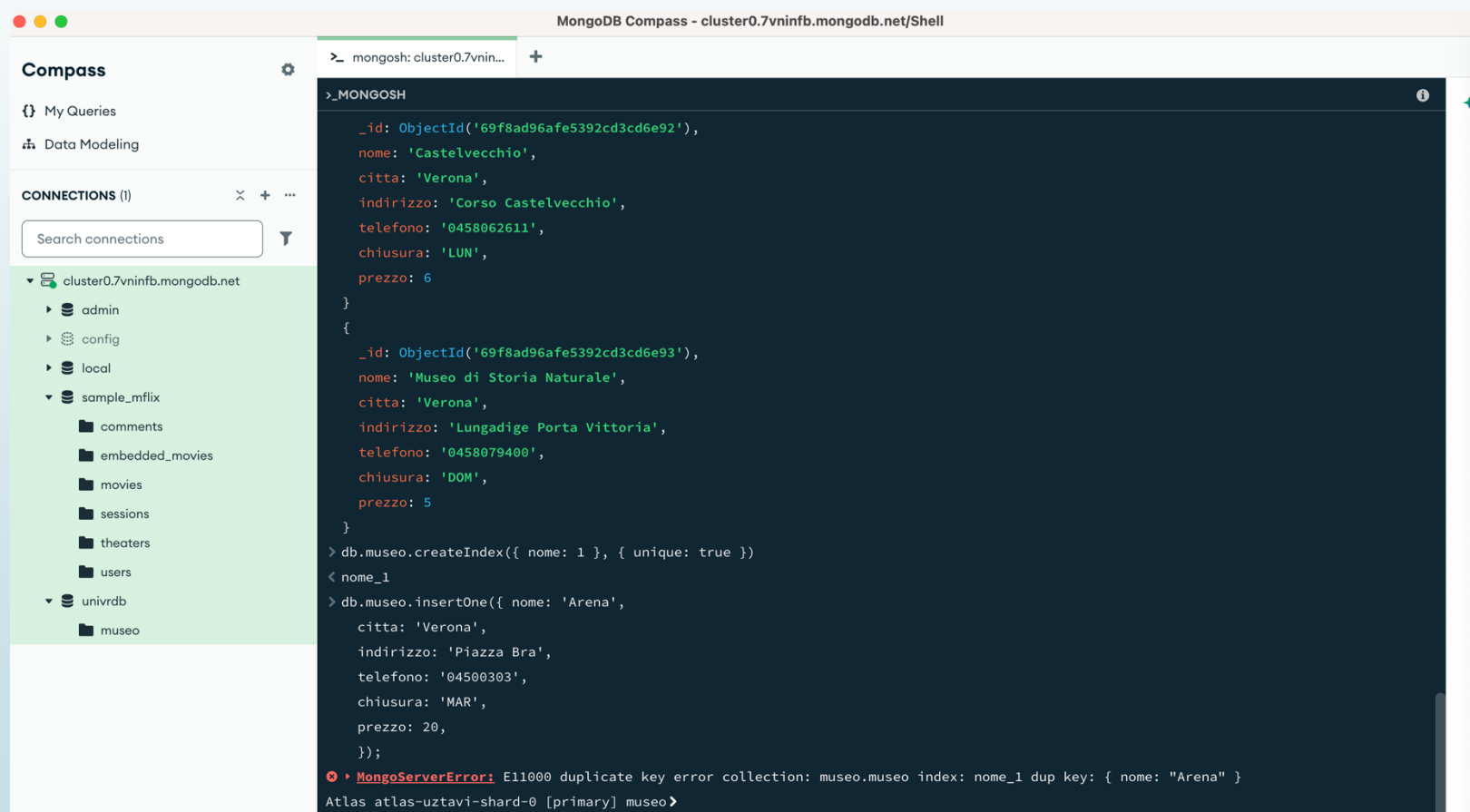
The main window displays the MongoDB Shell. The prompt is `> mongosh: cluster0.7vninfb.mongodb.net/Shell`. The shell shows the following commands and output:

```
> _MONGOSH
prezzo: 20
}
{
  _id: ObjectId('69f8ad96afe5392cd3cd6e92'),
  nome: 'Castelvecchio',
  citta: 'Verona',
  indirizzo: 'Corso Castelvecchio',
  telefono: '0458062611',
  chiusura: 'LUN',
  prezzo: 6
}
{
  _id: ObjectId('69f8ad96afe5392cd3cd6e93'),
  nome: 'Museo di Storia Naturale',
  citta: 'Verona',
  indirizzo: 'Lungadige Porta Vittoria',
  telefono: '0458079400',
  chiusura: 'DOM',
  prezzo: 5
}
> db.museo.createIndex({ nome: 1 }, { unique: true })
MongoServerError[DuplicateKey]: Index build failed: 52fb870c-9a74-473d-8337-314b12638aa2: Collection museo.museo ( fcb33c56-cf5f-4c5
> db.museo.deleteOne({ nome: "Arena" })
< {
  acknowledged: true,
  deletedCount: 1
}
Atlas atlas-uztavi-shard-0 [primary] museo>
```

# Inserimento duplicati



# Inserimento duplicati



The screenshot shows the MongoDB Compass application interface. On the left, the 'CONNECTIONS' panel lists the connected database 'cluster0.7vninfb.mongodb.net' and its collections, including 'museum'. The main panel displays the MongoDB shell output. The first two documents are inserted successfully. The third document, which has a 'nome' of 'Arena', causes a duplicate key error because an index named 'nome\_1' already exists on the 'nome' field. The error message is: 'MongoServerError: E11000 duplicate key error collection: museo.museo index: nome\_1 dup key: { nome: "Arena" }'.

```
MongoDB Compass - cluster0.7vninfb.mongodb.net/Shell
> mongosh: cluster0.7vninfb.mongodb.net/Shell

> _MONGOSH

{
  "_id": ObjectId('69f8ad96afe5392cd3cd6e92'),
  "nome": 'Castelvecchio',
  "citta": 'Verona',
  "indirizzo": 'Corso Castelvecchio',
  "telefono": '0458062611',
  "chiusura": 'LUN',
  "prezzo": 6
}

{
  "_id": ObjectId('69f8ad96afe5392cd3cd6e93'),
  "nome": 'Museo di Storia Naturale',
  "citta": 'Verona',
  "indirizzo": 'Lungadige Porta Vittoria',
  "telefono": '0458079400',
  "chiusura": 'DOM',
  "prezzo": 5
}

> db.museo.createIndex({ nome: 1 }, { unique: true })
< nome_1

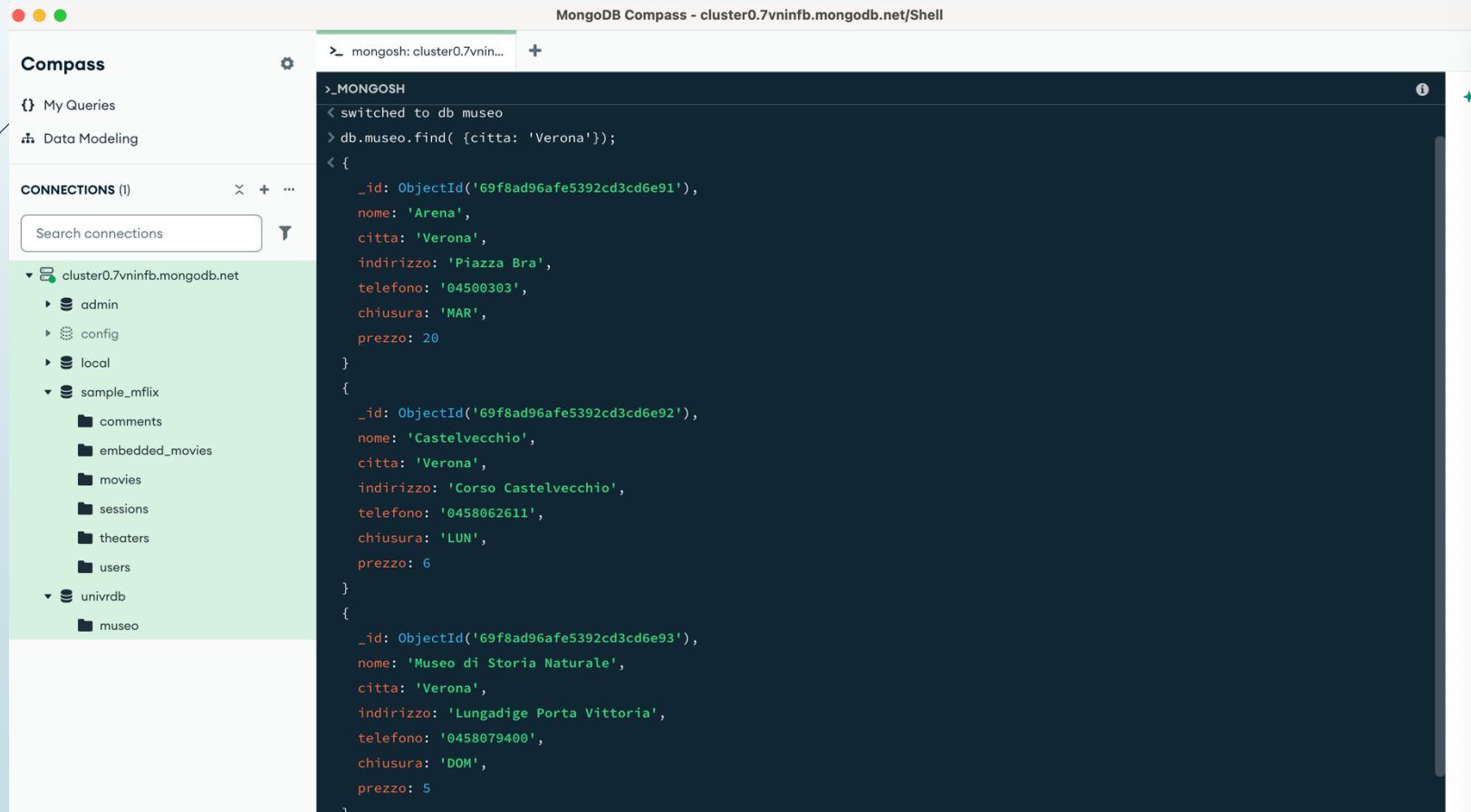
> db.museo.insertOne({ nome: 'Arena',
  citta: 'Verona',
  indirizzo: 'Piazza Bra',
  telefono: '04500303',
  chiusura: 'MAR',
  prezzo: 20,
});

MongoServerError: E11000 duplicate key error collection: museo.museo index: nome_1 dup key: { nome: "Arena" }
Atlas atlas-uztavi-shard-0 [primary] museo>
```

# Read: Ricerca

Ricerca di un documento in una collezione con un certo filtro

```
db.museo.find( {citta: 'Verona'});
```



Riceve in ingresso un oggetto JSON contenente il filtro (opzionale).

Restituisce gli oggetti JSON che soddisfano il filtro.



# Read: Ricerca

**db.museo.find({})** : Recupera tutti i documenti della collezione museo

**db.museo.find({ <field1>: <value1>, ... })** :Recupera tutti i documenti della collezione museo che soddisfano il filtro (Le condizioni sono in AND).

```
db.museo.find({  
  città: "Verona",  
  prezzo: 10  
})
```

# Read: Ricerca

## Operatori generici di confronto:

- { <field1>: { <operator1>: <value1> }, ... }

```
db.museo.find({  
  prezzo: { $lt: 10 }  
})
```

- \$in, \$nin (dentro una lista)

```
db.museo.find({  
  citta: { $in: ["Verona", "Roma"] }  
})
```

```
db.museo.find({  
  citta: { $nin: ["Verona"] }  
})
```

- \$eq, \$ne

```
db.museo.find({  
  prezzo: { $eq: 10 }  
})
```

- \$gt, \$gte, \$lt, \$lte



# Read: Ricerca

## Condizioni in OR

```
db.museo.find({
  $or: [
    { prezzo: { $lt: 30 } },
    { citta: { $regex: "^V" } }
  ]
});
```

## Condizioni in AND + OR

```
db.museo.find({
  nome: "Arena",
  $or: [
    { prezzo: { $lt: 30 } },
    { citta: { $regex: "^V" } }
  ]
});
```



# Ricerca con valori nulli

Ricerca con valori nulli

```
db.museo.find({ telefono: null });
```

```
db.museo.find({ telefono: { $ne: null } });
```

# Aggiornamento: Update

Aggiornare un documento: `updateOne()`

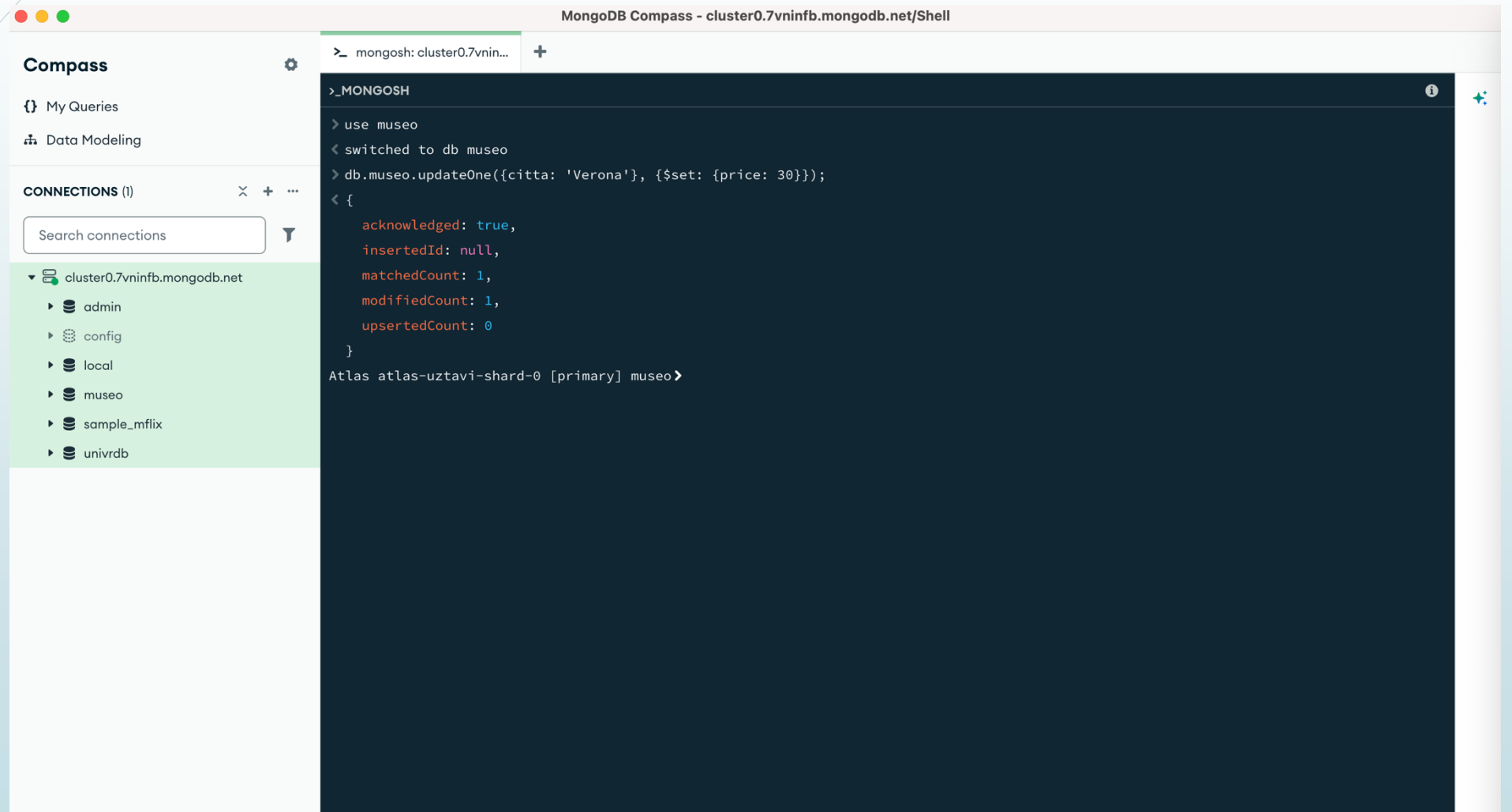
```
db.museo.updateOne ({citta: 'Verona'}, {$set: {price: 30}});
```

- L'oggetto JSON passato come primo parametro serve da filtro
- `$set`: permette di specificare un insieme di campi con il valore aggiornato
- Ritorna un oggetto JSON che contiene dei metadati:
  - `matchedCount`: notifica il numero di documenti che soddisfano il filtro
  - `modifiedCount`: notifica il numero di documenti che sono stati modificati

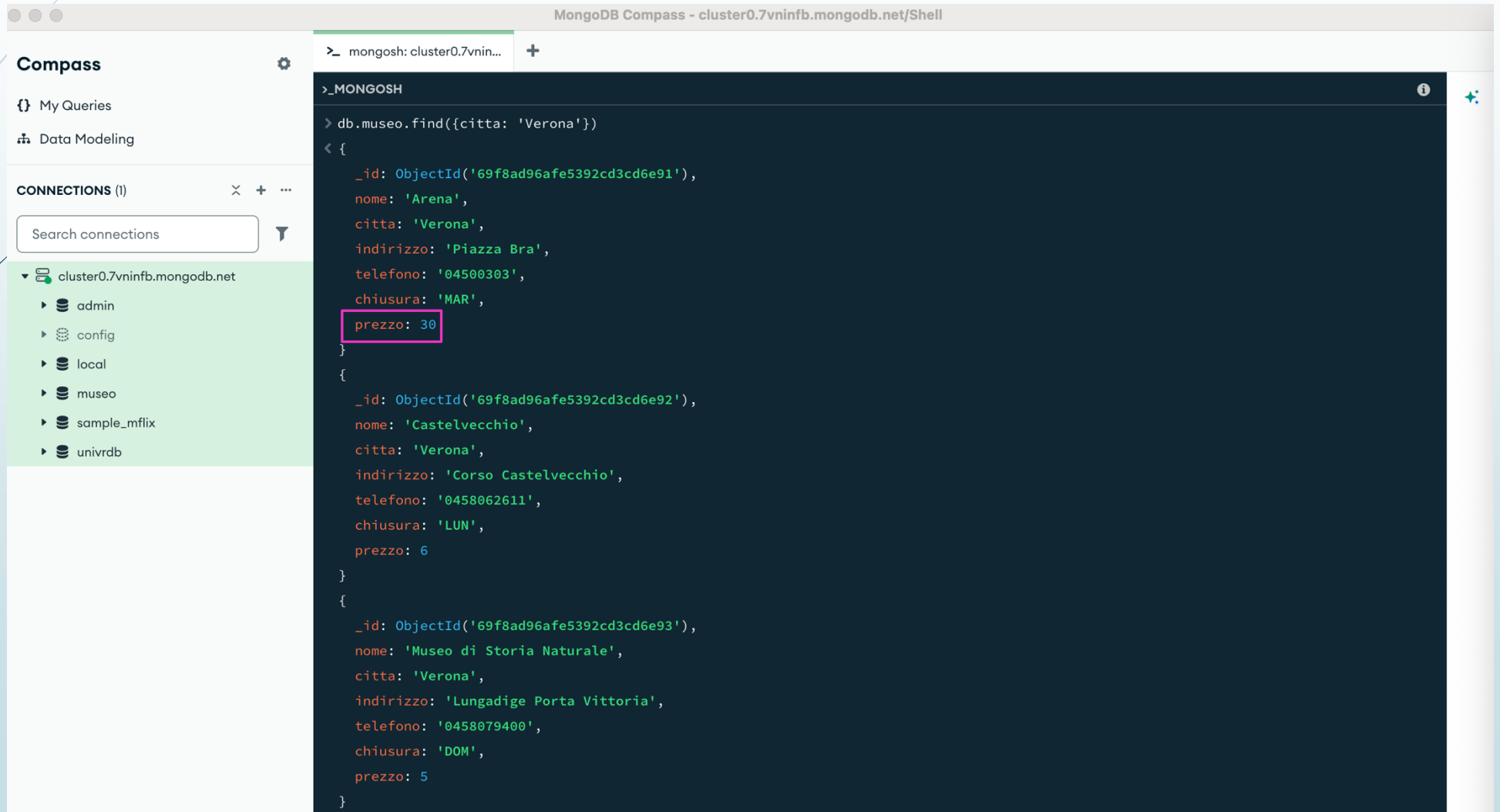
Con `updateOne()` viene modificato sempre un solo documento, il primo che soddisfa il criterio

# Aggiornamento: Update

```
db.museo.updateOne({citta: 'Verona'}, {$set: {prezzo: 30}});
```



# Aggiornamento: Update



The screenshot displays the MongoDB Compass application interface. On the left sidebar, the 'Compass' tab is active, showing 'My Queries' and 'Data Modeling' options. Below these, the 'CONNECTIONS (1)' section lists the connected cluster 'cluster0.7vninfb.mongodb.net'. Under this cluster, several databases are listed: 'admin', 'config', 'local', 'museo', 'sample\_mflix', and 'univddb'. The 'museo' database is selected. The main panel on the right shows the MongoDB Shell with a query executed: `> db.museo.find({citta: 'Verona'})`. The result is a JSON array of three documents. The first document, representing the 'Arena', has its 'prezzo' field (value 30) highlighted with a pink rectangular box. The other two documents represent 'Castelvecchio' (prezzo: 6) and 'Museo di Storia Naturale' (prezzo: 5).

```
>_MONGOSH
> db.museo.find({citta: 'Verona'})
< [
  {
    _id: ObjectId('69f8ad96afe5392cd3cd6e91'),
    nome: 'Arena',
    citta: 'Verona',
    indirizzo: 'Piazza Bra',
    telefono: '04500303',
    chiusura: 'MAR',
    prezzo: 30
  },
  {
    _id: ObjectId('69f8ad96afe5392cd3cd6e92'),
    nome: 'Castelvecchio',
    citta: 'Verona',
    indirizzo: 'Corso Castelvecchio',
    telefono: '0458062611',
    chiusura: 'LUN',
    prezzo: 6
  },
  {
    _id: ObjectId('69f8ad96afe5392cd3cd6e93'),
    nome: 'Museo di Storia Naturale',
    citta: 'Verona',
    indirizzo: 'Lungadige Porta Vittoria',
    telefono: '0458079400',
    chiusura: 'DOM',
    prezzo: 5
  }
]
```

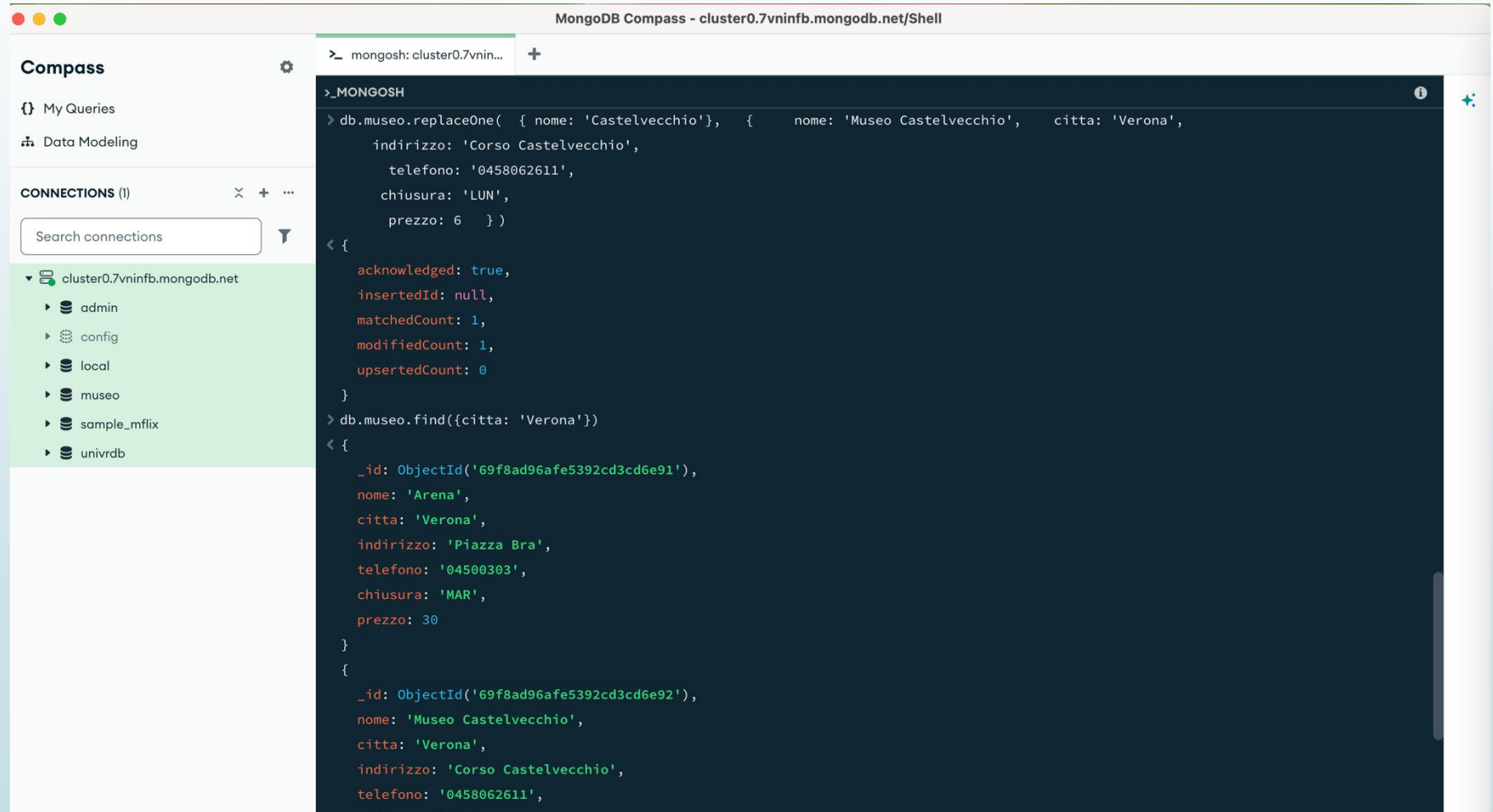
# Aggiornamento: Update

`db.collection.updateMany()`: Aggiorna più documenti

`db.museo.replaceOne()`: Sostituisce un singolo documento mantenendo inalterato l'`ObjectId`, vengono riportati solo i campi che appaiono nel `replaceOne()`

```
db.museo.replaceOne(  
  { nome: 'Castelvecchio'},  
  {  
    nome: 'Museo Castelvecchio',  
    citta: 'Verona',  
    indirizzo: 'Corso Castelvecchio',  
    telefono: '0458062611',  
    chiusura: 'LUN',  
    prezzo: 6  
  }  
)
```

# Aggiornamento: Update



The screenshot shows the MongoDB Compass application interface. On the left, the 'Compass' sidebar is visible with sections for 'My Queries', 'Data Modeling', and 'CONNECTIONS (1)'. Under 'CONNECTIONS', a cluster named 'cluster0.7vninfb.mongodb.net' is expanded, showing databases: 'admin', 'config', 'local', 'museo', 'sample\_mflix', and 'univddb'. The 'museo' database is selected. The main panel on the right displays the 'mongo: cluster0.7vninfb.mongodb.net/Shell' window. It shows a JavaScript command being executed in the mongo shell:

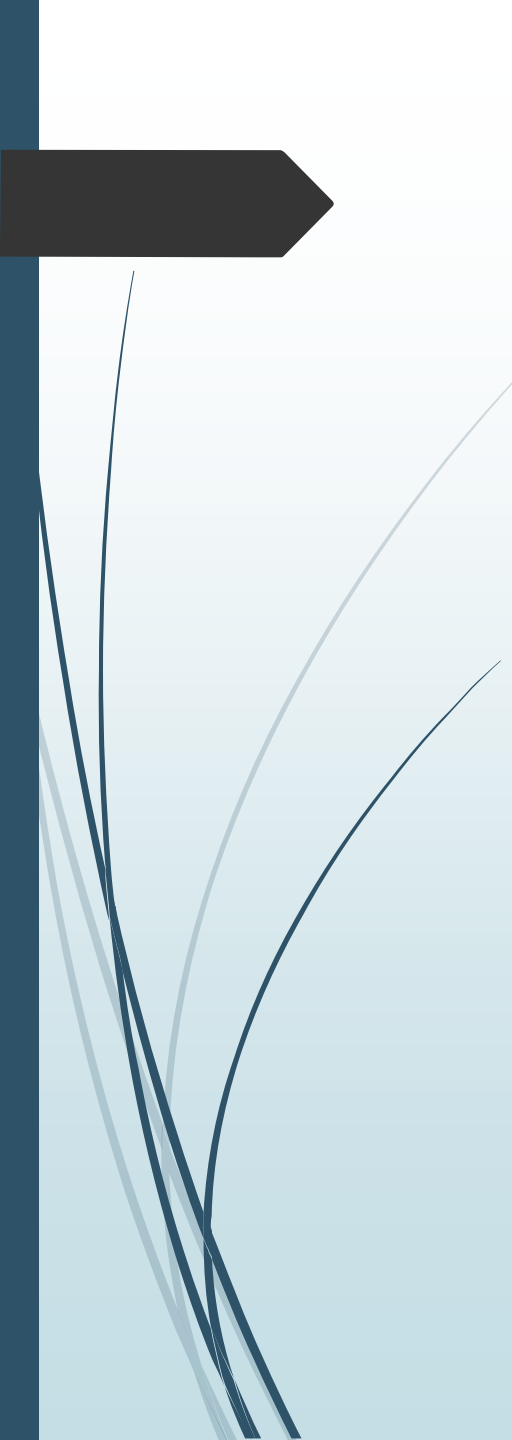
```
> db.museo.replaceOne( { nome: 'Castelvecchio'}, { nome: 'Museo Castelvecchio', citta: 'Verona',  
  indirizzo: 'Corso Castelvecchio',  
  telefono: '0458062611',  
  chiusura: 'LUN',  
  prezzo: 6  })
```

The output of the command is displayed below the prompt:

```
< {  
  acknowledged: true,  
  insertedId: null,  
  matchedCount: 1,  
  modifiedCount: 1,  
  upsertedCount: 0  
}
```

Below the output, the result of a find operation is shown:

```
> db.museo.find({citta: 'Verona'})  
< {  
  _id: ObjectId('69f8ad96afe5392cd3cd6e91'),  
  nome: 'Arena',  
  citta: 'Verona',  
  indirizzo: 'Piazza Bra',  
  telefono: '04500303',  
  chiusura: 'MAR',  
  prezzo: 30  
}  
{  
  _id: ObjectId('69f8ad96afe5392cd3cd6e92'),  
  nome: 'Museo Castelvecchio',  
  citta: 'Verona',  
  indirizzo: 'Corso Castelvecchio',  
  telefono: '0458062611',  
  chiusura: 'LUN',  
  prezzo: 6  
}
```



# Cancellazione: Delete

Come abbiamo visto prima, la cancellazione di un documento da una collezione avviene con:

```
db.museo.deleteOne({citta: 'Verona'});
```

Parametro: oggetto JSON che permette di selezionare particolari documenti.

`deleteOne()` cancella sempre il primo documento che rispetta la condizione di filtro.

Ritorna un oggetto JSON con dei metadati, tra cui: `deletedCount`: numero di documenti che sono stati cancellati

`deleteMany()` cancella tutti i documenti su cui trova una corrispondenza.

# Schema in Documenti MongoDB

- Il modello relazionale predilige la proprietà di normalizzazione del dato: una stessa informazione è presente una sola volta all'interno della base di dati e le relazioni permettono di creare collegamenti da informazioni (evitare ridondanza!).
- In un database NoSQL è prevista la possibilità di avere ridondanza e strutture de-normalizzate per incrementare le prestazioni e garantire flessibilità (dato non strutturato o semi-strutturato).
- I documenti MongoDB consentono il concetto di **incapsulamento**: la proprietà di un documento può essere un altro documento.
  - L'incapsulamento riduce la necessità di JOIN e facilita il recupero dell'informazione.
  - La migliore struttura ed il miglior criterio di incapsulamento dipende dai requisiti dell'applicazione (tipico modo di interrogare l'informazione).

# Incapsulamento

```
{
  _id: ObjectId("f34303859302abcd12345678"),
  titolo: "Viaggio al centro della Terra",
  anno_publicazione: 1864,
  generi: ["Avventura", "Fantascienza"],
  autore: {
    nome: "Jules",
    cognome: "Verne",
    anno_nascita: 1828,
    nazionalita: "Francese",
    biografia: "Un autore pionieristico nel genere della fantascienza"
  },
  valutazione: 4.5,
  copie_vendute: 1500000
}
```

Cosa succede se ho un altro libro  
dello stesso autore?



# Riferimento

Attraverso i riferimenti è possibile creare dei collegamenti tra i dati tramite «link» o «puntatori».

// Course Collection

```
{  
  _id: ObjectId("d04940004ncdd"),  
  course_name: "Introduction to Artificial Intelligence",  
  professor_id: ObjectId("4590f394ks"), // riferimento  
  duration: "6 months",  
  credits: 6  
}
```

// Professor Collection

```
{  
  _id: ObjectId("4590f394ks"),  
  first_name: "Alan",  
  last_name: "Turing",  
  department: "Computer science",  
  publications: 45  
}
```

# Denormalizzazione

```
// Course Collection
{
  "_id": ObjectId("d04940004ncdd"),
  "course_name": "Introduction to Artificial Intelligence",
  "professor_id": ObjectId("4590f394ks"),
  "professor_name": "Alan Turin",
  "duration": "6 months",
  "credits" : 6
}

// Professor Collection
{
  "_id": ObjectId("4590f394ks"),
  "first_name": "Alan",
  "last_name": "Turing",
  "department": "Computer science",
  "publications" : 45
}
```

Approccio a metà tra incapsulamento e riferimento che consente di velocizzare l'accesso a certe informazioni.

# MongoDB Formato e tipi di dato

MongoDB memorizza le informazioni nel formato BSON (Binary JSON) che è del tutto simile a JSON ma offre una più ampia varietà di tipi base e delle ottimizzazioni per la memorizzazione e la ricerca.

Tipi di dato supportati da BSON:

- “\_id”: ObjectId(“407f1f77bcf86cd799439012”),
- “doubleField”: 55.4,
- “stringField”: “Hello, world!”,
- “objectField”: {“name”: “John”, “age”: 34},
- “arrayField”: [1,2,3,4,5],
- “binaryField”: BinData(0, “binary-content”),
- “booleanField”: true,
- “dataField”: ISODate(“2024-05-12T10:00:00Z”),



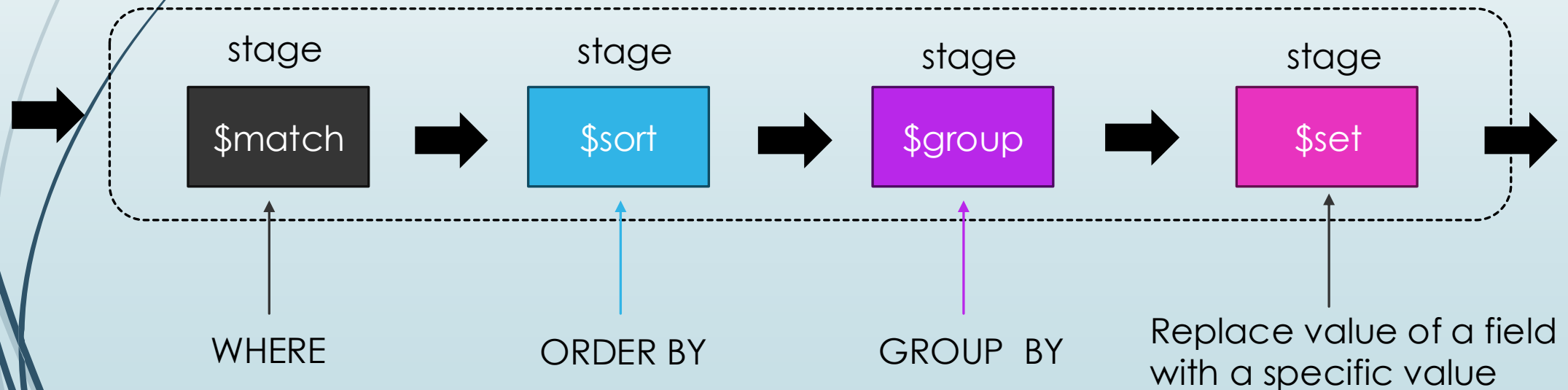
# MongoDB Formato e tipi di dato

- "nullField": null,
- "regexField": /^abc/,
- "javascriptField": Code( "function() { print( 'success!'); }" ),
- "javascriptWithScopeField": Code( "return this.x + this.y; {x: 3, y: 5}" ),
- "intField": 42,
- "timestampField": Timestamp(1601410400, 1),
- "int64Field": NumberLong( 9876543210 ),
- "decimal128Field": NumberDecimal( "0.123456789" ),
- "minKeyField": MinKey(),
- "maxKeyField": MaxKey(),

# MongoDB: Aggregazioni

Le aggregazioni sono uno strumento che consente di combinare più trasformazioni e interrogazioni, definendo una pipeline di processamento.

In questa pipeline, ad ogni passo viene passato all'operazione corrente un insieme di documenti, mentre l'insieme ottenuto dalla trasformazione è passato all'operazione successiva.



# MongoDB: Aggregazioni

Le aggregazioni sono uno strumento che consente di combinare più trasformazioni e interrogazioni, definendo una pipeline di processamento.

```
db.museo.aggregate([  
  { $match: { citta: "Verona" } },  
  {  
    $group: {  
      _id: "$citta",  
      mediaPrezzo: { $avg: "$prezzo" }  
    }  
  }  
])
```

media prezzo dei musei per città

# MongoDB: operatori

| SQL clause, term, function | MongoDB aggregation operators |
|----------------------------|-------------------------------|
| WHERE                      | \$match                       |
| GROUP BY                   | \$group                       |
| HAVING                     | \$match                       |
| SELECT                     | \$project                     |
| LIMIT                      | \$limit                       |
| OFFSET                     | \$skip                        |
| ORDER BY                   | \$sort                        |
| SUM()                      | \$sum                         |
| COUNT()                    | \$sum AND \$sortByCount       |
| JOIN                       | \$lookup                      |
| SELECT INTO NEW_TABLE      | \$out                         |
| MERGE INTO TABLE           | \$merge                       |
| UNION ALL                  | \$unionWith                   |

# MongoDB: Join

L'operatore di aggregazione \$lookup permette di eseguire un **left outer join** rispetto ad una collezione della stessa base di dati.

```
{
  $lookup:
  {
    from: <collection to join>,
    localField: <field from the input documents>,
    foreignField: <field from the documents of the "from" collection>,
    let: { <var_1>: <expression>, ..., <var_n>: <expression> },
    pipeline: [ <pipeline to run> ],
    as: <output array field>
  }
}
```

Collezione esterna presente nello stesso DB

Attributo del documento corrente da usare per il JOIN

Attributo del documento esterno da usare per il JOIN

Nome dell'attributo che conterrà l'elenco dei documenti collegati



# MongoDB: Join

```
db.museo.aggregate([
  {
    $lookup: {
      from: "citta",
      localField: "citta_id",
      foreignField: "_id",
      as: "info_citta"
    }
  }
])
```